

白盒测试及测试用例设计

东南大学计算机科学与工程学院

李必信

目录

- 一 白盒测试概念
- 二 静态白盒测试
- 三 动态白盒测试
 - ✓ 基于控制流测试
 - ✓ 基本路径测试
 - ✓ 基于数据流测试

一 白盒测试概念

1. 什么是白黑测试？
2. 为什么需要白盒测试？
3. 白盒测试有什么特点？
4. 如何实施白盒测试活动？

Testing Tactics



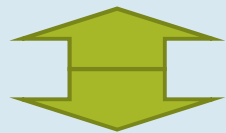
- Tests based on *spec*
- Test covers as much *specified* behavior as possible

- Tests based on *code*
- Test covers as much *implemented* behavior as possible

1. 什么是白黑测试？

“白盒测试=结构化测试”，两个概念是等价的

白盒测试两个特点：（1）基于代码；（2）尽可能覆盖实现的行为。



黑盒测试两个特点：（1）基于规约；（2）尽可能覆盖定义的行为。

2. 为什么需要白盒测试?

Why Structural?



- If a part of the program is never executed, a defect may loom in that part
A “part” can be a statement, function, transition. condition...

- Attractive because automated

2. 为什么需要白盒测试?

Why Structural?



- **Complements functional tests**
Run functional tests first, then measure what is missing
- **Can cover low-level details missed in high-level specification**

2. 为什么需要白盒测试? ...

- (1) 确保每段代码都被执行, 避免相应的缺陷;
- (2) 是黑盒测试/功能测试的补充;
- (3) 能覆盖高层规范说明中的忽视的底层细节;
- (4) 更容易进行自动化测试。

.....

3. 白盒测试有什么特点？

白盒测试：是一种基于**源程序**或**代码**的测试方法。依据**源程序**或**代码结构与逻辑**生成测试用例，以尽可能多地发现并修改源程序错误。

白盒测试分为**静态**和**动态**两种类型。

其它称谓：**结构测试 逻辑驱动测试 基于程序的测试**

4. 如何实施白盒测试活动？

■ 实施者

单元测试阶段：一般由开发人员进行。

集成测试阶段：一般由测试人员和开发人员共同完成。

■ 步骤

动态：

(1)程序图(CFG,PDG); (2)生成测试用例;(3)执行测试;(4)分析覆盖标准;(5)判定测试结果

静态：

4. 如何实施白盒测试活动? ...

■ 进入和退出条件

进入条件: 编码开始阶段

退出条件:

- (1) 完成测试计划 (满足一定覆盖率)
- (2) 发现并修正了错误
- (3) 预算和开发时间

4. 如何实施白盒测试活动? ...

静态白盒测试
动态白盒测试

二 静态白盒测试

1. 什么是静态白盒测试？
2. 为什么需要进行静态白盒测试？
3. 静态白盒测试具体方法是什么？

1. 什么是静态白盒测试？

早些时候，软件界大多数人都持有一个想法，即编写程序仅仅是为了提供给机器执行，并不是供人们阅读，软件测试的唯一方法就是在计算机上执行它。20世纪70年代早期，一些程序员最先意识到阅读代码对于构成完善的软件测试和调试手段的价值，通过他们的努力，原有的观念开始发生变化。

今天，不是所有的测试人员都要阅读代码，但阅读程序代码作为测试工作的一部分，这个观念已经得到了认同。相关的原因有：（1）软件规模和复杂度；（2）开发团队规模；（3）开发时限；（4）团队的文化和技术背景。

--- 《软件测试艺术》（ The Art of Software Testing ）

1. 什么是静态白盒测试?

■ 定义

静态白盒测试：在不执行软件的条件下有条理地仔细审查软件设计、体系结构和代码，从而找出软件缺陷的过程，有时称为结构化分析。

实施静态白盒测试的理由：

1. 尽早发现软件缺陷.
2. 为后继测试中设计测试用例提供思路.

2. 为什么需要进行静态白盒测试？

- 理解理由1：
 - NASA软件工程实验室(Basili and Selby 1987): 阅读代码每小时能够检测出的缺陷数要比测试高出80%。
 - Ackerman等人1989: 用测试来检测缺陷的成本是人工检查的6倍。
 - IBM 1995: 检查发现一个错误需要3.5个工作时, 测试需要花费15-25个工作时。

——《代码大全2》中文版

- 理解理由2: 测试人员通过旁听检查人员的审查评论, 可对软件有一定理解, 更好确定软件缺陷的范围。

2. 为什么需要进行静态白盒测试？

静态白盒测试优点

- (1) 可发现某些机器发现不了的错误。
- (2) 利用不同人对代码的不同观点。
- (3) 对照设计，确保程序能完成预期功能。
- (4) 不但能检测出错误，还可以尝试确定错误根源。
- (5) 节约计算机资源，但以增加人工成本为代价。
- (6) 尽早发现缺陷，避免后期缺陷修复造成的巨大压力。

3. 静态白盒测试具体方法是什么？

- 技术评审technical review
- 代码/文档阅读code/document reading
- 走查/通查walk through
- 专项检查inspection
- 代码/数据/文档审计code/data/document audit

三 动态白盒测试

- 动态白盒测试简介
- 基于控制流测试方法
 - ✓ 逻辑覆盖测试
 - ✓ 基本路径测试
 - ✓ 循环处理方法
- 基于数据流测试方法

动态白盒测试简介

■特点

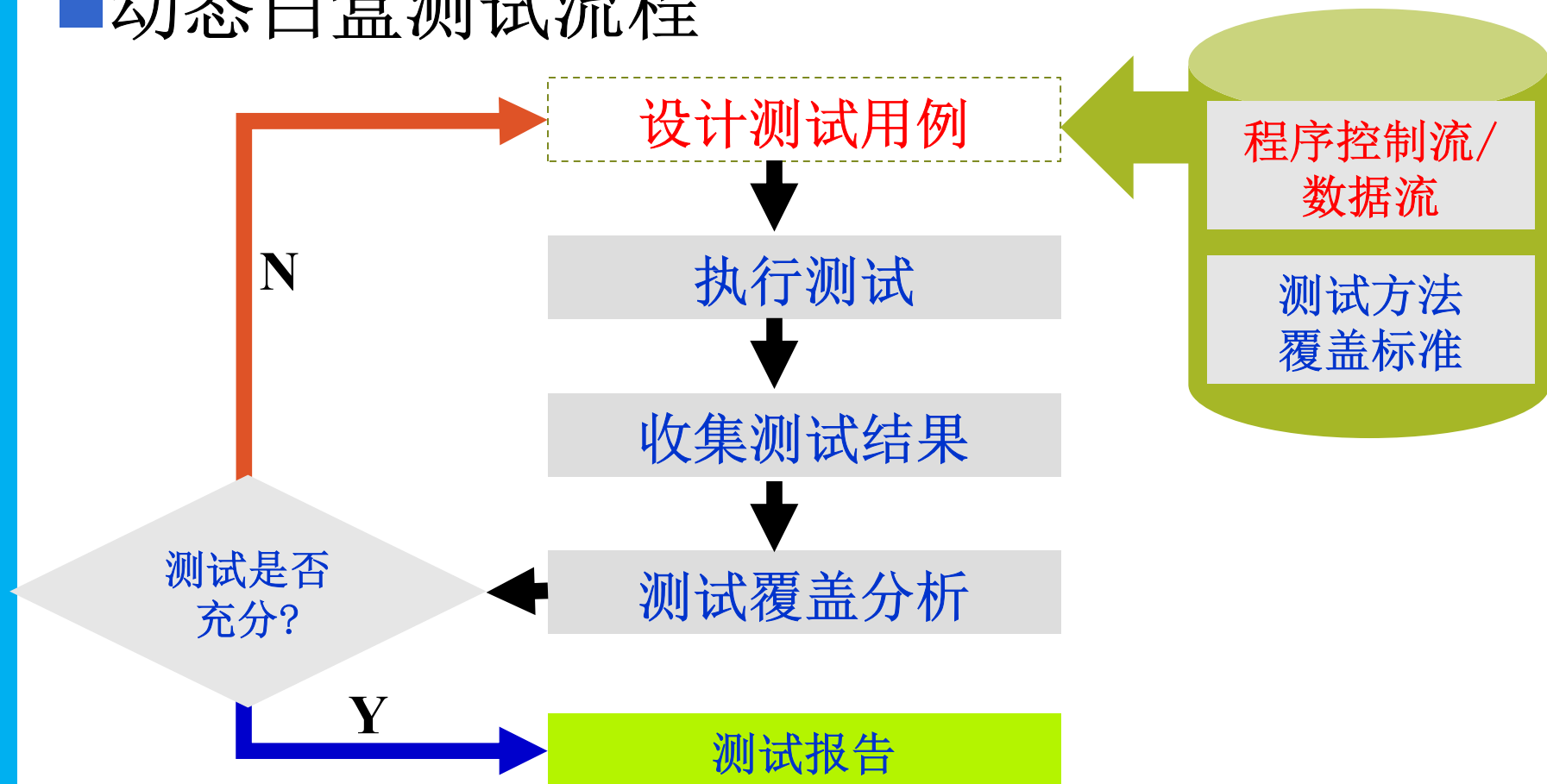
- 不但要提供软件源代码，还要提供可执行程序，测试过程需要在计算机上执行程序。

■动态白盒测试检查是否存在缺陷是通过…

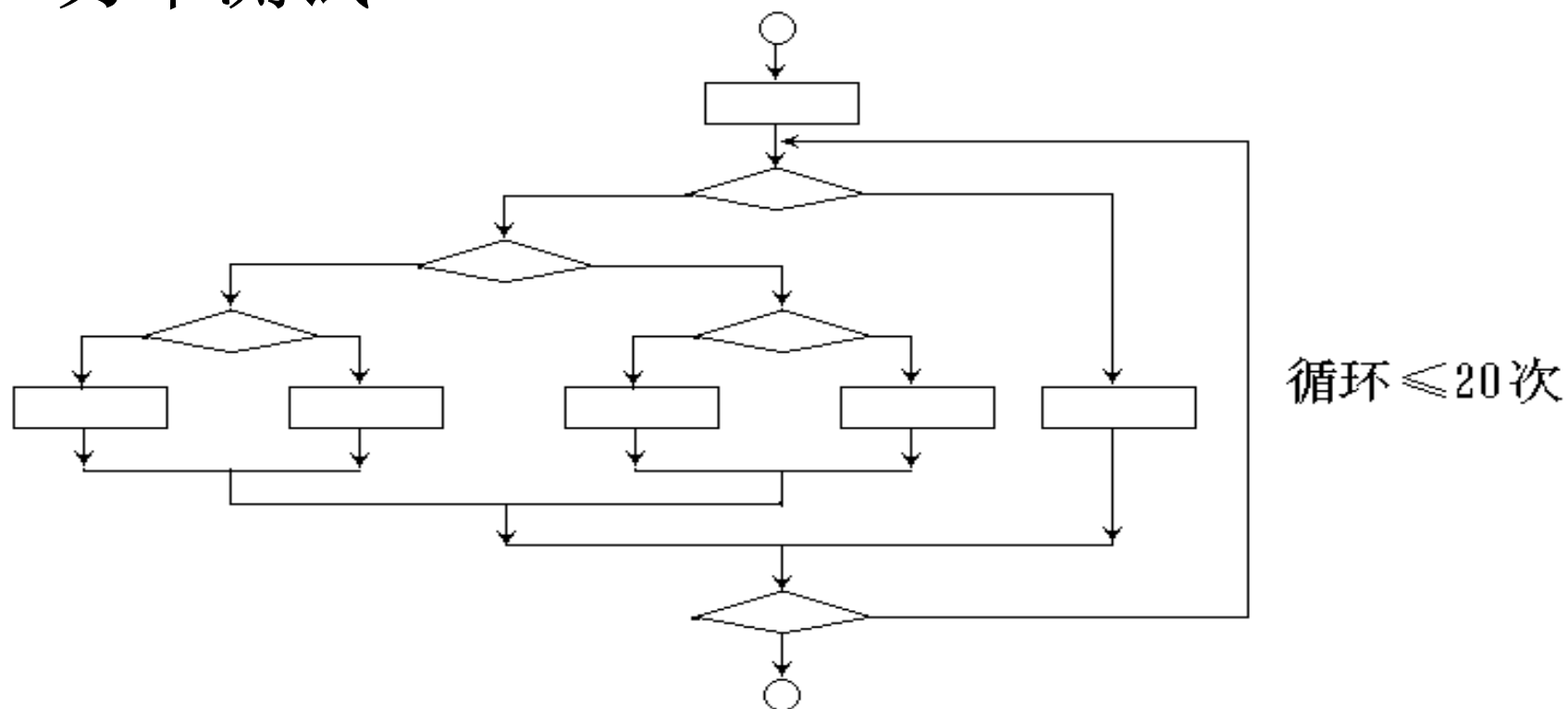
- 对程序模块中的所有独立执行路径至少测试执行一次
- 对所有逻辑判定的取值（“真”与“假”）都至少测试执行一次
- 在上下边界及可操作范围内测试运行所有循环
- 测试执行内部数据结构来检查其有效性
-

动态白盒测试简介...

■ 动态白盒测试流程



■ 穷举测试？



对一个具有多重选择和循环嵌套的程序，不同的路径数目可能是天文数字。例如一个小程序的流程图如上所示，它包括了一个执行**20**次的循环，其循环体有五个分支。这个循环的不同执行路径数达 **5^{20}** 条，如果对每一条路径进行测试需要**1**毫秒，那么即使一年工作 **365×24** 小时，要想把所有路径测试完，大约需**3170**年。

动态白盒测试简介...

■ 覆盖标准简介

- **覆盖标准的意义**：对“测试执行到何时才是充分的？”的定量回答。
- **覆盖标准的作用**：测试软件的一种度量标准，描述程序源代码被测试的**程度**。
- **重要**：一种测试技术通常有一种对应的覆盖标准。

动态白盒测试简介...

■ 动态白盒测试方法

□ 基于控制流的方法

- 1. 语句测试——语句覆盖
- 2. 条件测试——判定覆盖、条件覆盖等
- 3. 路径测试——路径覆盖

□ 基于数据流的方法

- 4. 数据流测试

基于控制流的测试

基于控制流的方法一种逻辑覆盖测试

白盒测试要求对被测程序结构特性做到一定程度的覆盖

常用覆盖标准

1	语句覆盖	保证程序中的每条语句都执行一遍
2	判定覆盖	保证每个判断取True和False至少一次
3	条件覆盖	保证每个判断中的每个条件的取值至少满足一次
4	判定条件覆盖	保证每个条件和由条件组成的判断的取值...
5	条件组合覆盖	保证每个条件的取值组合至少出现一次...
6	路径覆盖	覆盖程序中所有可能路径

无论哪种覆盖方法，都无法绝对保证程序的正确性

基于控制流的方法…

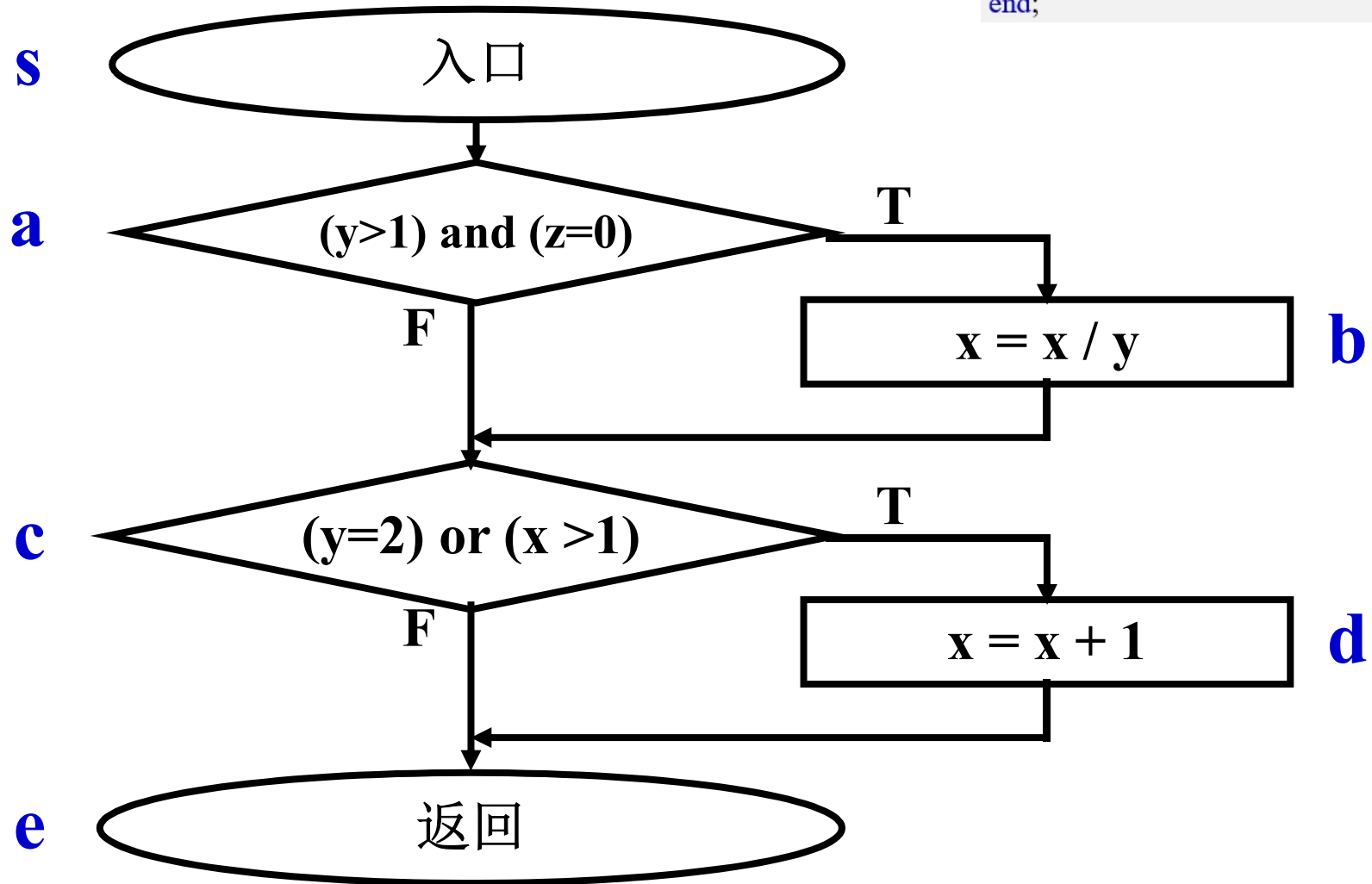
对下列Pascal语言写的子程序进行测试

```
procedure example(y,z:real;var x:real);  
begin  
    if (y>1) and (z=0) then x:=x/y;  
    if (y=2) or (x>1) then x:=x+1;  
end;
```

该子程序接受x、y、z的值，并将计算结果x的值返回给调用程序。

与该子程序对应的流程图如下：

```
procedure example(y,z:real;var x:real);  
begin  
  if (y>1) and (z=0) then x:=x/y;  
  if (y=2) or (x>1) then x:=x+1;  
end;
```



基于控制流的方法…

1.语句覆盖

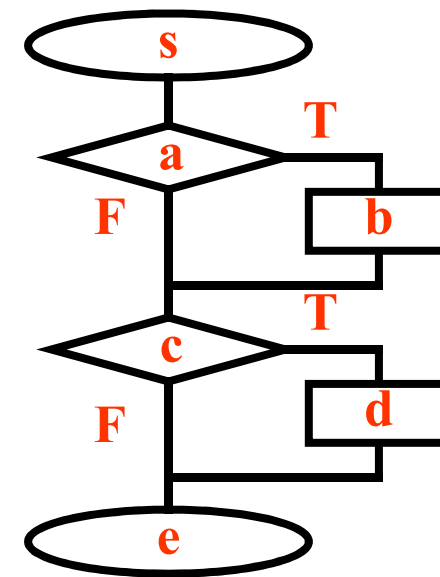
■思想：语句覆盖(**Statement Coverage**)是指选择足够的测试用例，使得运行这些测试用例时，被测程序的每条可执行语句都至少执行一次

其它称谓：所有节点覆盖

- 欲使每条语句都执行一次，只需执行路径 **sabcde** 即可。

- 测试用例 (y, z, x):
 - 输入 (2, 0, 4)
 - 输出 (2, 0, 3)
- 是否能发现遗漏边

```
procedure example(y,z:real;var x:real);  
begin  
  if (y>1) and (z=0) then x:=x/y;  
  if (y=2) or (x>1) then x:=x+1;  
end;
```



a: (y>1) and (z=0)

c: (y=2) or (x>1)

基于控制流的方法…

100%的语句覆盖可能很困难

- (1) 处理错误的代码片段
- (2) 小概率事件(软件开发中的恶作剧)
- (3) 不可达代码

```
if (x>y && y>z && z>x){  
    cout<<“This should never happen!”;  
}
```

基于控制流的方法…

即使是100%语句覆盖，有时也无法发现某些严重问题！

```
int* p=NULL;  
if (condition) {  
    p=&variable;  
}  
  
*p=123;
```

int* p=NULL; 定义指针变量**p**，初始指向为null

p=&variable; 使得**p**指向**variable**的地址

***p=123** 该指向的地址内填123

基于控制流的方法…

2.判定覆盖

思想：判定覆盖（也称分支覆盖）是指选择足够的测试用例，使得运行这些测试用例时，被测程序的每个判定的所有可能结果都至少执行一次（即判定的每个分支至少经过一次）

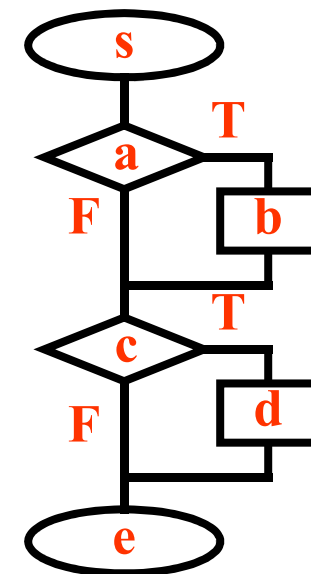
```

procedure example(y,z:real;var x:real);
begin
  if (y>1) and (z=0) then x:=x/y;
  if (y=2) or (x>1) then x:=x+1;
end;

```

- 保证每个判断取真假至少一次
(即判定的每个分支至少经过一次)
- 路径: $sacde + sabce$; $sabcde + sace$
- 测试用例 (y, z, x) :

测试数据	预期结果	路径	a	c
2, 1, 1	2, 1, 2	sacde	F	T
3, 0, 3	3, 0, 1	sabce	T	F



a: $(y>1) \text{ and } (z=0)$
c: $(y=2) \text{ or } (x>1)$

基于控制流的方法…

判定覆盖(**Decision Coverage**): 保证每个判断至少获得一次 **True**和**False**

其它称谓: 分支覆盖 所有边覆盖

优点: 简单! 包含语句覆盖并避免了语句覆盖的问题!

缺点: 忽略了表达式内的条件, 不能发现每个条件的错误。

基于控制流的方法...

例子：控制结构的执行不依赖条件 **condition2**

```
if (condition1 && (condition1 || condition2))  
    statement1;  
else  
    statement2;
```

基于控制流的方法...

3. 条件覆盖

思想：保证每个判断中的每个条件的取值至少满足一次

例： $x > 3$ OR $y < 5$

两个原子条件： $x > 3$ $y < 5$

测试用例1： $x=4$ $y=8$ $\text{true OR false} = \text{true}$

测试用例2： $x=3$ $y=3$ $\text{false OR true} = \text{true}$

- 选择足够的测试用例，使得运行这些测试用例时，被测程序的*每个判定中的每个条件的所有可能结果都至少出现一次*
- 测试用例 (y, z, x):

条件覆盖不能保证程序所有分支都被执行

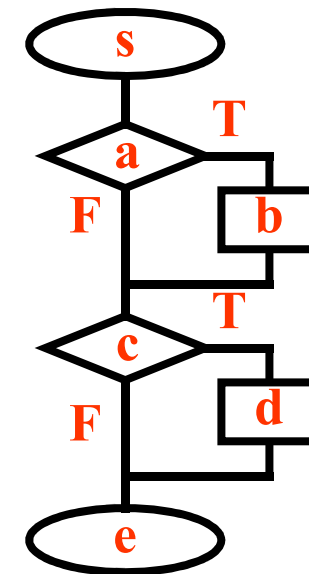
测试数据	预期结果	路径	覆盖条件	a	c
1, 0, 3	1, 0, 4	sacde	F T F T	F	T
2, 1, 1	2, 1, 2	sacde	T F T F	F	T



```

procedure example(y,z:real;var x:real);
begin
  if (y>1) and (z=0) then x:=x/y;
  if (y=2) or (x>1) then x:=x+1;
end;

```



a: (y>1) and (z=0)

c: (y=2) or (x>1)

基于控制流的方法...

4.判定条件覆盖

思想：保证每个条件和由条件组成的判断的取值（取真取假各一次）

例： $x > 3 \text{ OR } y < 5$

(1) $x=6 \ y=3$ $T \text{ OR } T = T$ (可省略)

(2) $x=6 \ y=8$ $T \text{ OR } F = T$ (保留)

(3) $x=2 \ y=3$ $F \text{ OR } T = T$ (保留)

(4) $x=2 \ y=8$ $F \text{ OR } F = F$ (保留)

$T \text{ OR}$
 $F \text{ AND}$

- 被测程序的每个判定的所有可能结果都至少执行一次，并且，每个判定中的每个条件的所有可能结果都至少出现一次。
- 测试用例 (y, z, x):

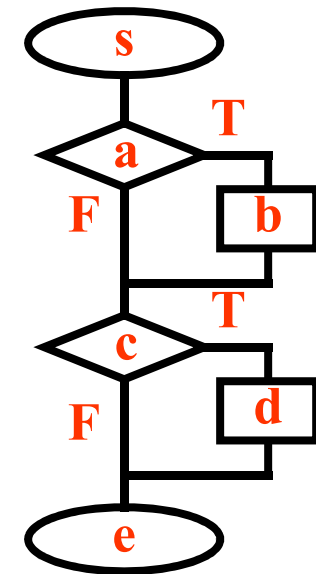
测试数据	预期结果	路径	覆盖条件	a	c
1, 0, 3	1, 0, 4	sacde	F T F T	F	T
2, 1, 1	2, 1, 2	sacde	T F T F	F	T
3, 0, 0	3, 0, 1	sabce	T T F F	T	F



```

procedure example(y,z:real;var x:real);
begin
  if (y>1) and (z=0) then x:=x/y;
  if (y=2) or (x>1) then x:=x+1;
end;

```



a: (y>1) and (z=0)

c: (y=2) or (x>1)

基于控制流的方法...

- 对于条件的每个组合，首先必须确定哪个组合（测试用例）对发现错误是敏感的？哪个组合会产生错误屏蔽？造成错误被屏蔽的组合在测试过程中可以不考虑，因此需要考虑的测试用例数量要少一些。
- 错误屏蔽：指原子条件取值改变不会影响判定结果，因此该条件上的取值错误是不可见的。

例如

`condition1 && (condition1 || condition2)`

`condition2`的取值是不可见的

基于控制流的方法...

ID	原子条件			判定
	a	b	c	a&&(b c)
1	T	T	T	T
2	F	F	F	F

100%覆盖

100%覆盖但无法
检测出运算符错误

ID	原子条件			判定	
	a	b	c	a (b c)	a&&(b&&c)
1	T	T	T	T	T
2	F	F	F	F	F

基于控制流的方法...

5.条件组合覆盖

思想(1): 选择足够的测试用例, 使得每个判定中条件的各种可能组合都至少出现一次

例: $x > 3 \text{ OR } y < 5$

(1) $x=6 \ y=3$ **T OR T = T**

(2) $x=6 \ y=8$ **T OR F = T**

(3) $x=2 \ y=3$ **F OR T = T**

(4) $x=2 \ y=8$ **F OR F = F**

基于控制流的方法...

例: $x \geq 3 \text{ AND } x < 5$

(1) $x=4$: $T \text{ OR } T = T$

(2) $x=8$ $T \text{ OR } F = T$

(3) $x=1$ $F \text{ OR } T = T$

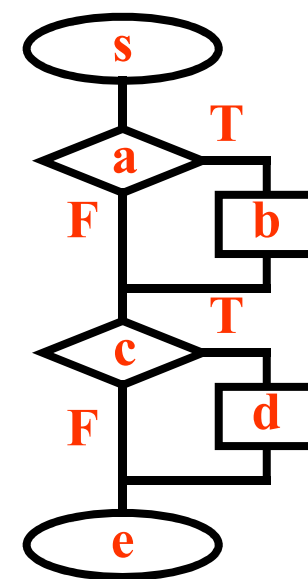
(4) $x=?$ $F \text{ OR } F = F$

某些条件组合是不可能的!

- 测试用例 (y, z, x) :

测试数据	预期结果	覆盖路径	覆盖的条件
2, 0, 4	2, 0, 3	sabcde	T T T T
2, 1, 1	2, 1, 2	sacde	T F T F
1, 0, 3	1, 0, 4	sacde	F T F T
1, 1, 1	1, 1, 1	sace	F F F F

是否经过sabcde路径? 【TTFF、TTFT、TTTF】



a: (y>1) and (z=0)

c: (y=2) or (x>1)

思想(2): 所有条件的所有可能取值组合至少出现一次

a: $(y > 1) \text{ and } (z = 0)$

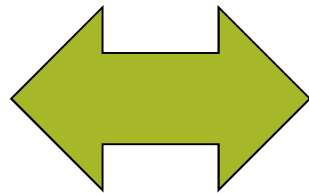
c: $(y = 2) \text{ or } (x > 1)$

■ TT

■ TF

■ FT

■ FF

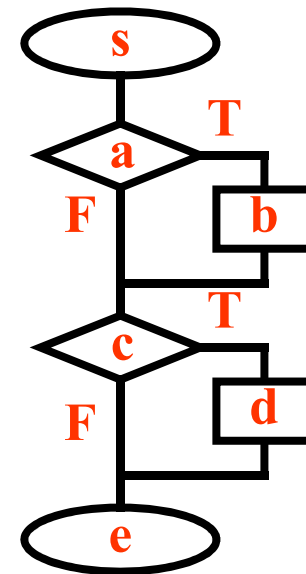


■ TT

■ TF

■ FT

■ FF

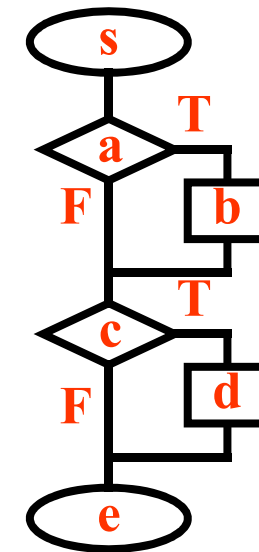


代价昂贵: 2^n 组合数目, n 为原子条件数

TTTT、TTTF、TTFT、TFTT、
FTTT、TTFF、TFFT、FFTT、
TFTF、FTFT、FTTF、TFFF、
FTFF、FFTF、FFFT、FFFF

- 哪些条件组合不存在？
- 需要多少测试用例？【错误屏蔽】
- 条件组合覆盖所有的路径吗？

```
procedure example(y,z:real;var x:real);  
begin  
  if (y>1) and (z=0) then x:=x/y;  
  if (y=2) or (x>1) then x:=x+1;  
end;
```



a: (y>1) and (z=0)

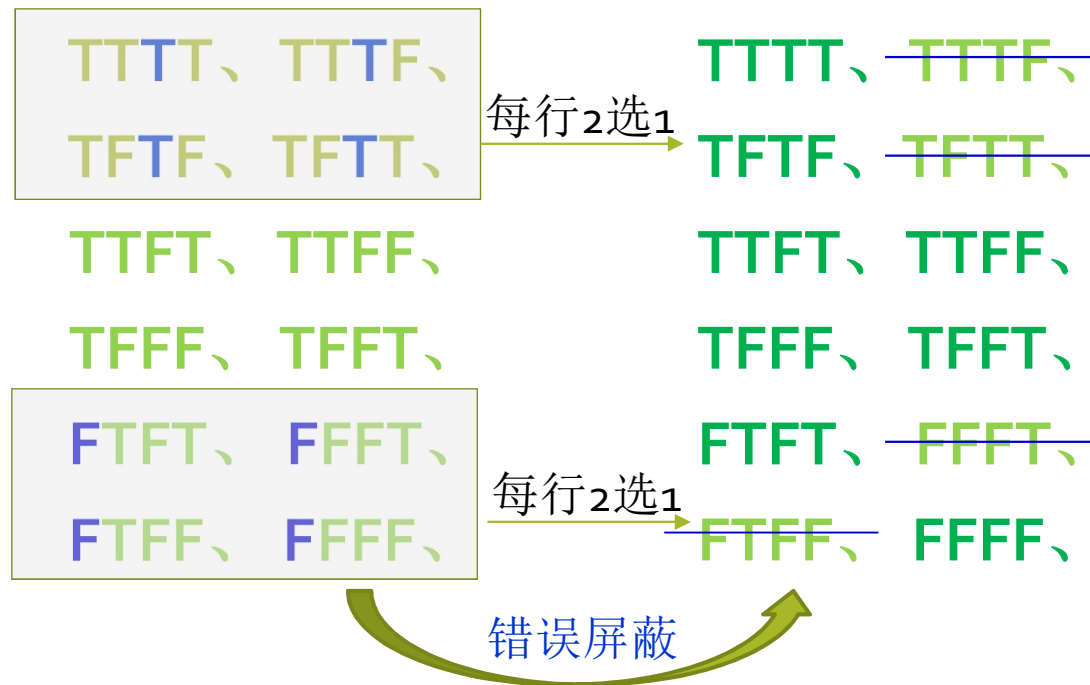
c: (y=2) or (x>1)

条件组合不存在

• TTTT、 TTTF、
TTFT、 TFTT、
FTTT、 TTFF、
TFFT、 FFTT、
TFTF、 FTFT、
FTTF、 TFFF、
FTFF、 FFTF、
FFFT、 FFFF

TTTT、 TTTF、
TFTF、 TFTT、
TTFT、 TTFF、
TFFF、 TFFT、
FTFT、 FFTT、
FTFF、 FFFF、

a: $(y > 1)$ and $(z = 0)$ **T** OR
c: $(y = 2)$ or $(x > 1)$ **F** AND

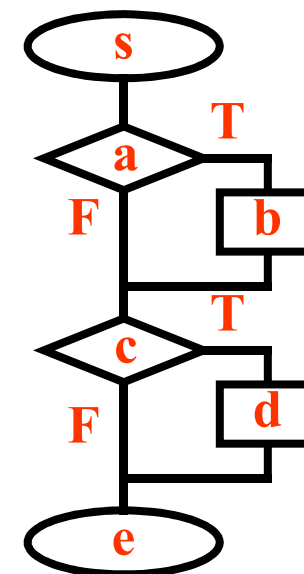


测试数据	预期结果	路径	覆盖条件	a	c
2, 0, 4		sabcde	T T T T	T	T
2, 1, 1		sacde	T F T F	F	T
3, 0, 2		sacde	T T F T	F	T
3, 0, 0		sabce	T T F F	T	F
3, 1, 0		sace	T F F F	F	F
3, 1, 2		sacde	T F F T	F	T
1, 0, 2		sacde	F T F T	F	T
1, 1, 0		sace	F F F F	F	F

```

procedure example(y,z:real;var x:real);
begin
  if (y>1) and (z=0) then x:=x/y;
  if (y=2) or (x>1) then x:=x+1;
end;

```



a: (y>1) and (z=0)

c: (y=2) or (x>1)

基于控制流的方法…

6. 路径覆盖

路径覆盖(Path Coverage)思想：选择足够的测试用例，保证每条可能执行到的路径都至少经过一次（如果程序中包含环路，则要求每条环路至少经过一次）

优点：相对彻底的测试

缺点：1) 路径条数可能以分支的指数级增加： 2^x (x为分支次数)；
（2）不可达路径存在

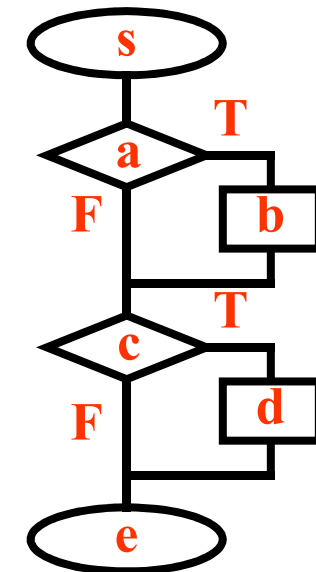
```

procedure example(y,z:real;var x:real);
begin
  if (y>1) and (z=0) then x:=x/y;
  if (y=2) or (x>1) then x:=x+1;
end;

```

- 路径测试用例 (y, z, x):

测试数据	预期结果	路径	覆盖条件	a	c
2, 0, 4	2, 0, 3	sabcde	T T T T	T	T
1, 0, 1	1, 1, 1	sace	F T F F	F	F
1, 0, 2	1, 0, 3	sacde	F T F T	F	T
3, 0, 3	3, 0, 1	sabce	T T F F	T	F



a: (y>1) and (z=0)

c: (y=2) or (x>1)

基于控制流的方法…

- 路径覆盖实际上考虑了程序中各种判定结果的所有可能组合，但它未必能覆盖判定中条件结果的各种可能情况。
- 它是一种比较强的覆盖标准，但不能替代条件覆盖和条件组合覆盖标准。

基于控制流的方法...



路径覆盖VS条件组合覆盖VS条件判定覆盖VS条件覆盖
VS判定覆盖VS语句覆盖？

路径覆盖

条件组合覆盖

判定条件覆盖

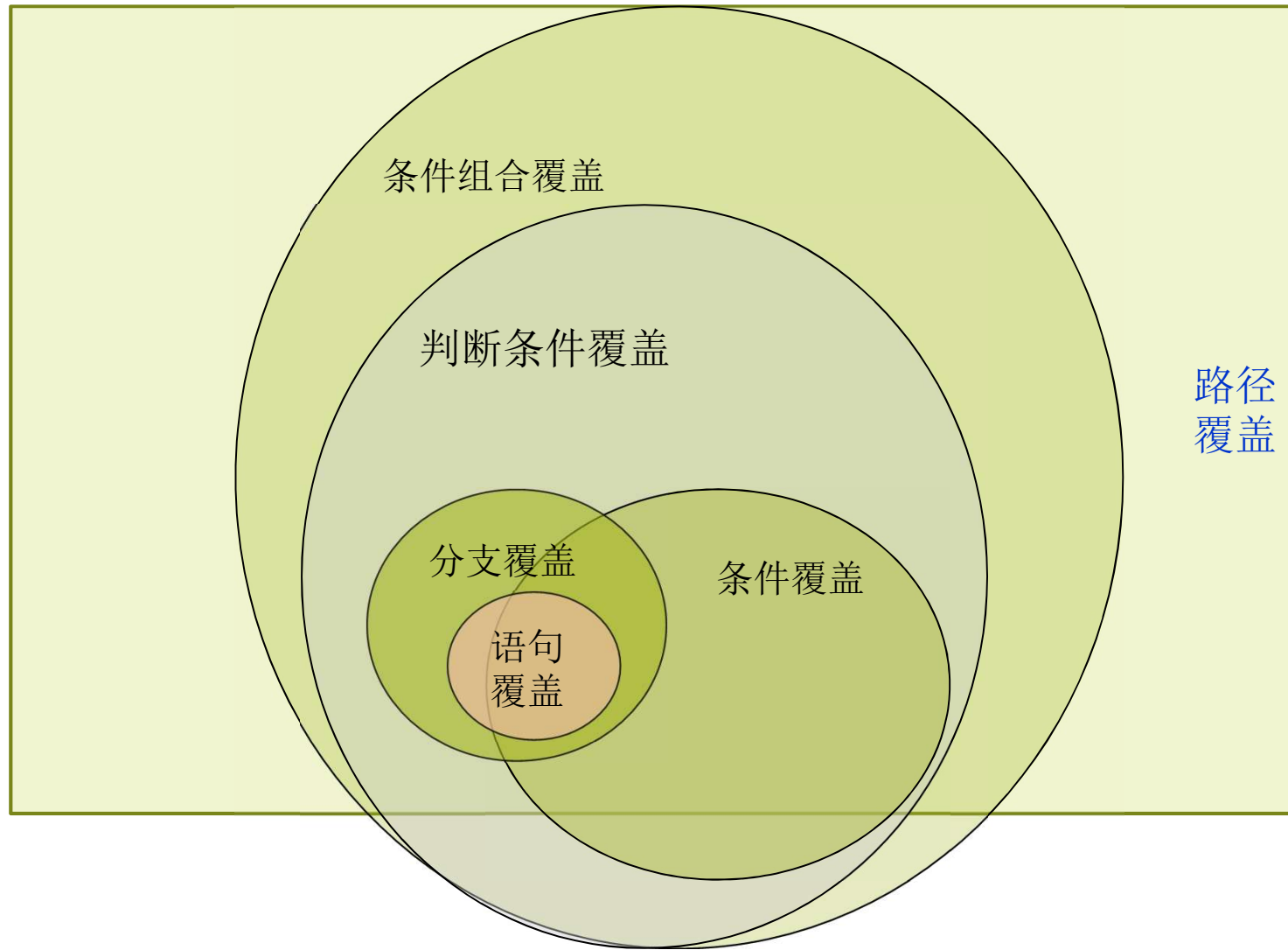
条件覆盖

判定覆盖

语句覆盖

覆盖能力



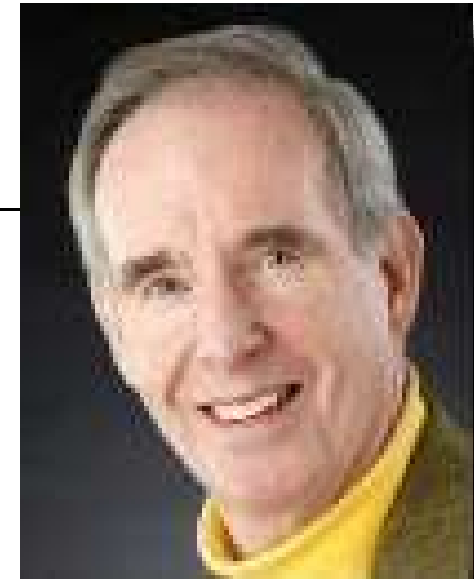


基于控制流的测试

—基本路径覆盖

基本路径测试

思想：寻找基本路径，根据基本路径构造测试用例，保证每条基本路径至少执行一次。



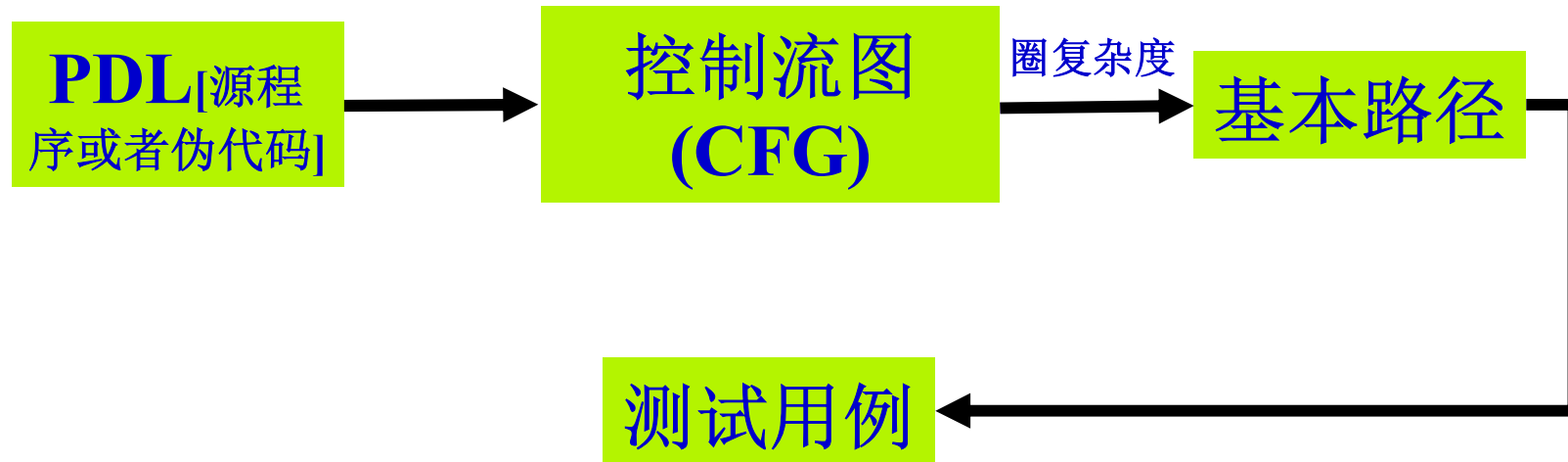
Thomas J. McCabe



McCabe(麦凯布). A Complexity Measure, IEEE Trans. on SE, 1976

基本路径测试...

■ 测试用例设计过程



实际上，这就是将程序流程抽象为图（连通图），通过对图的问题求解得到程序的基本路径。

基本路径测试...

■ 控制流图（**control flow graph, CFG**）

解释： 控制流图用来描述程序中的逻辑控制流

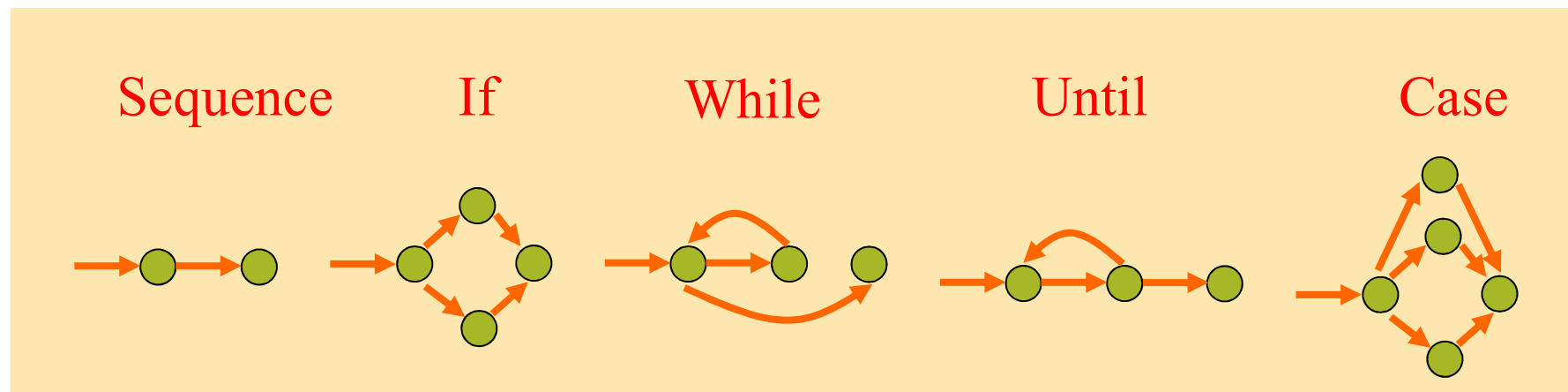
组成：

- 结点：表示一个或多个过程语句
- 边：表示控制流
- 域：由边和结点限定的区间

基本路径测试...

■ 控制流图

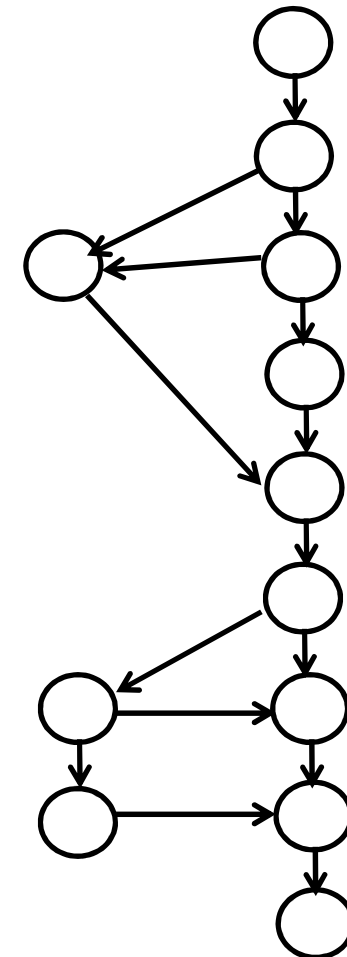
控制流图基本结构



练习：根据PDL画出控制流图

PDL: program design language, 伪代码
化简: 2和3合并, 7和8合并, 6和11删除

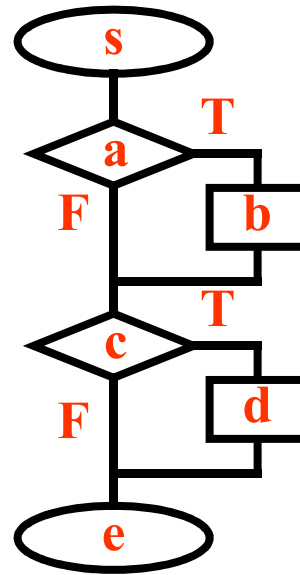
```
1:  start
    input (a,b,c,d)
2:  if (a>0)
3:      and (b>0)
4:  then x=a+b
5:  else x=a-b
6:  endif
7:  if (c>a)
8:      or (d<b)
9:  then y=c-d
10: else y=c+d
11: endif
12: print (x,y)
    stop
```



基本路径测试...

■ 基本路径

定义: 又称独立程序路径, 是指任何一条贯穿程序的路径, **该路径至少包含一条不同于其他路径的边。**



```
procedure example(y,z:real;var x:real);  
begin  
    if (y>1) and (z=0) then x:=x/y;  
    if (y=2) or (x>1) then x:=x+1;  
end;
```

正确: sabcde、sabce、sace
或者 sabcde、sacde、sace
或者 sabcde、sabce、sacde

~~错误~~: sabcde、sabce、sacde、sace

两个核心问题

- 如何确定基本路径的条数？

- McCable复杂度度量

- 基本路径有什么特性？

- ① 基本路径集合中路径条数唯一，基本路径可以不一样
 - ② 其他路径可以通过基本路径运算得到

基本路径测试...

■ 基本路径（独立程序路径）

基本路径举例：

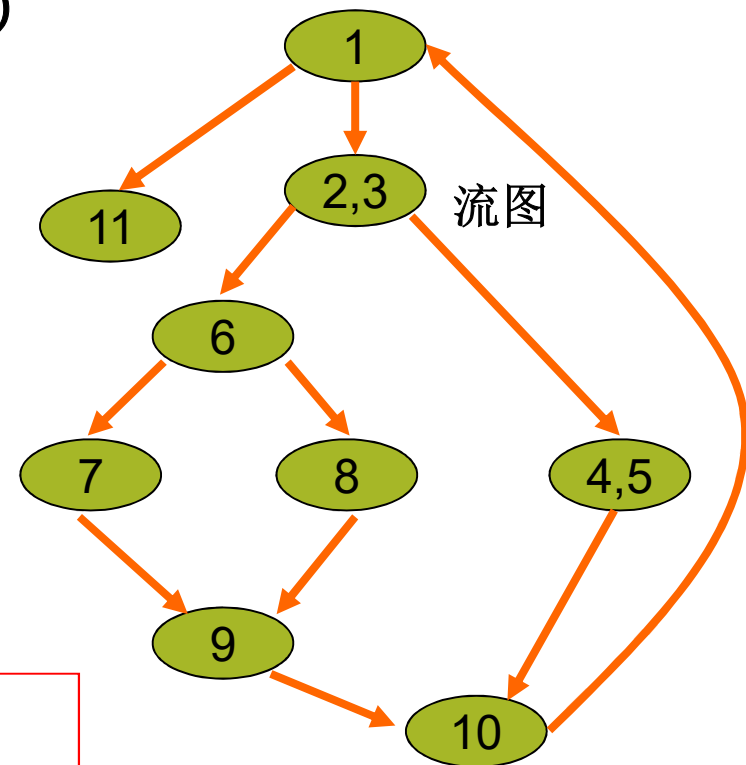
L1: 1-11

L2: 1-2-3-4-5-10-1-11

L3: 1-2-3-6-8-9-10-1-11

L4: 1-2-3-6-7-9-10-1-11

问题：可以有几条基本路径？



基本路径测试...

■基本路径（独立程序路径）

问题：一个程序对应的基本路径是什么？

■基本路径的度量——圈复杂度

圈复杂度度量基本路径数，是所有语句被执行一次所需测试用例数的上限之一，即执行所有基本路径即可实现100%语句覆盖。所有基本路径覆盖 $\neq$ 分支覆盖！

具有最高圈复杂度的模块蕴含错误的
可能性最大，是测试中关注的焦点。

基本路径测试...

圈复杂度	对应含义
1-10	代码编写良好，有很好的可测试性，维护成本低。
10-20	中等复杂度，可测试性中等，维护成本中等。
20-40	非常复杂，可测试性低，维护成本高。
>40	不可测。

基本路径测试...

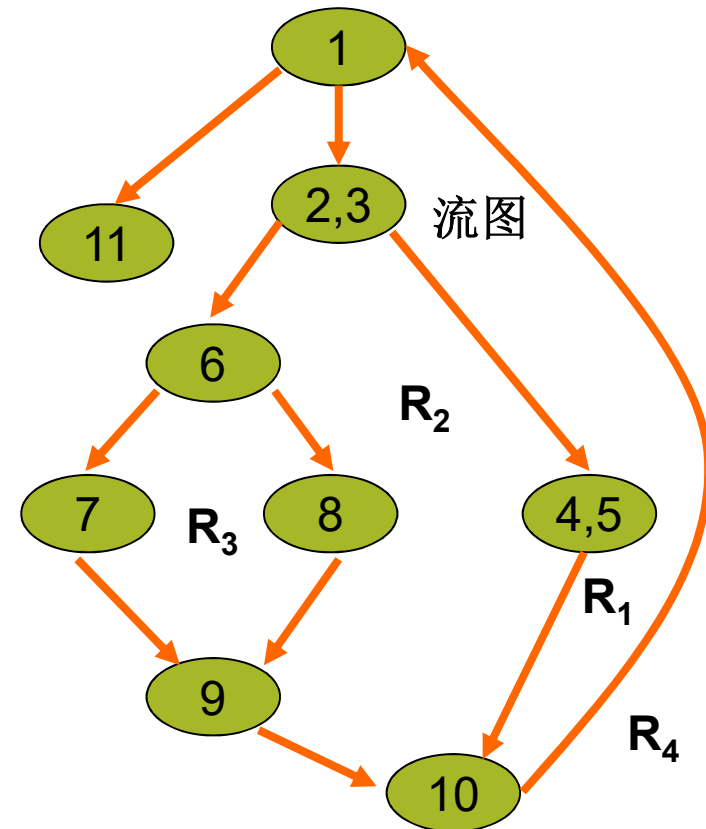
■ 基本路径（独立程序路径）

■ 圈复杂度的计算

- 方法1：等于区域的数量
- 方法2： $V(G)=E-N+2$, E 为边， N 为结点
- 方法3： $V(G)=P+1$, P 为判定结点数

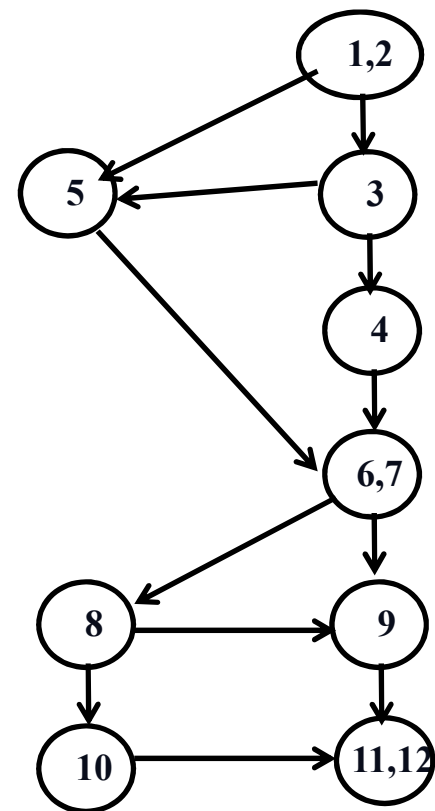
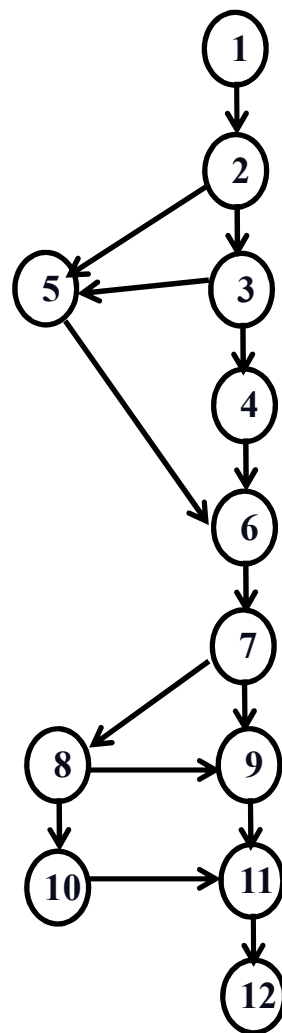
■ 实例

- $V(G)=4$
- $V(G)=11-9+2=4$
- $V(G)=3+1=4$



根据PDL画出控制流图

```
1: start
    input (a,b,c,d)
2: if (a>0)
3:     and (b>0)
4: then x=a+b
5: else x=a-b
6: endif
7: if (c>a)
8:     or (d<b)
9: then y=c-d
10: else y=c+d
11: endif
12: print (x,y)
    stop
```



顺序节点的合并并不影响圈复杂度分析，也不影响基本路径获取。

基本路径测试...

■基本路径（独立程序路径）

■基本路径集合

$P_1: 1 \rightarrow 11$

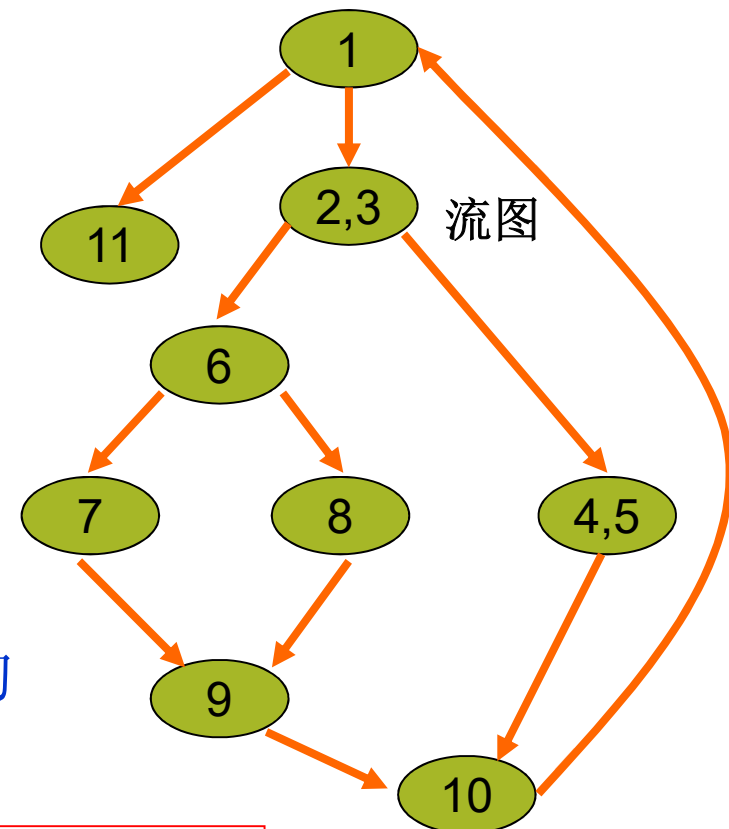
$P_2: 1 \rightarrow 2, 3 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 10 \rightarrow 1 \rightarrow 11$

$P_3: 1 \rightarrow 2, 3 \rightarrow 4, 5 \rightarrow 10 \rightarrow 1 \rightarrow 11$

$P_4: 1 \rightarrow 2, 3 \rightarrow 6 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 1 \rightarrow 11$

新路径特点：贯穿程序；引入了新的处理语句或新判断【增加了新的边】

基本路径集合并不唯一，但路径数是确定的！



基本路径测试...

■ 基本路径集寻找算法？

Step1: 确认从入口到出口的最短路径。

Step2: 从入口到第1个未被先后评估为真和假两种结果的条件语句。

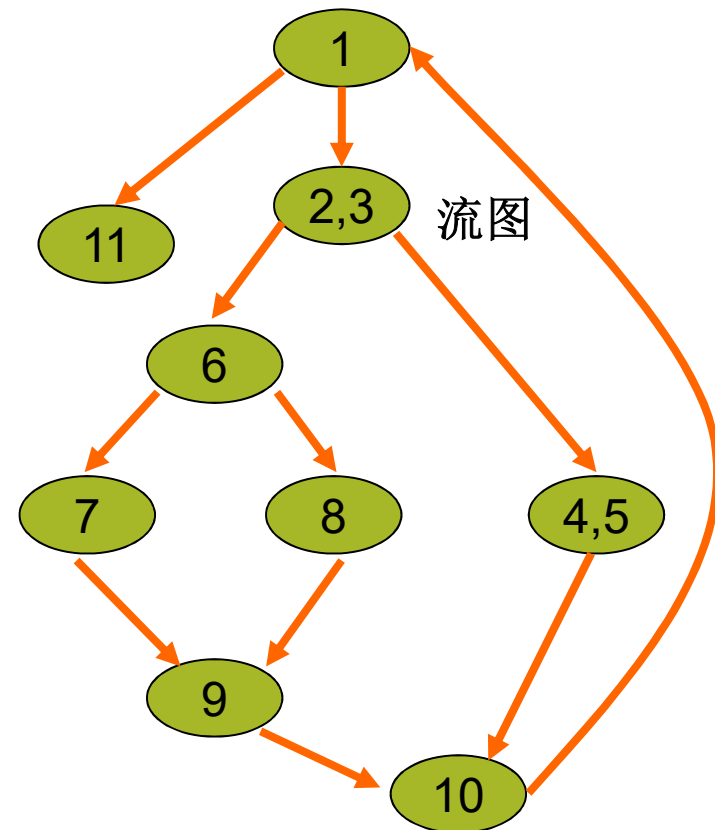
Step3: 改变该条件语句的判断值。

Step4: 按最短路径从这个条件语句到出口。

Step5: 重复步骤2-5, 直到所有基本路径都被找到。

基本路径测试...

```
void Func(int nPosX, int nPosY) {  
  1 while (nPosX > 0) {  
    2     int nSum = nPosX + nPosY;  
    3     if (nSum > 1) {  
      4         nPosX--;  
      5         nPosY--;  
    }  
    else {  
      6,7     if (nSum < -1) nPosX -= 2;  
      8     else nPosX -= 4;  
    }  
    9 }  
    10 } // end of while  
    11 }
```



基本路径测试...

P₁: 1→11

✓ 用例1: nPosX 取-1, nPosY取任意值

P₂: 1→2, 3→6→7→9→10→1→11

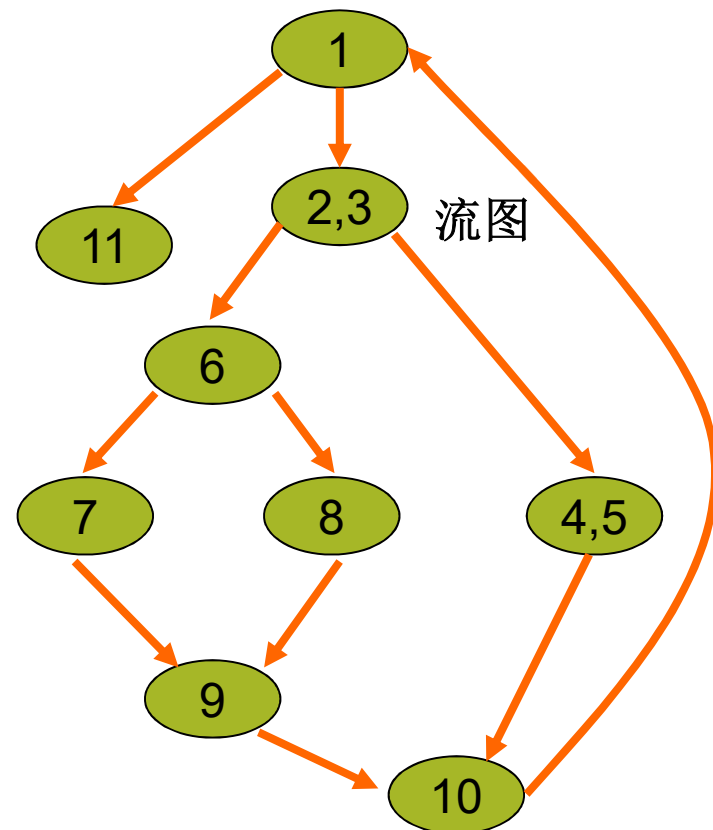
✓ 用例2: nPosX 取1, nPosY取1

P₃: 1→2, 3→4, 5→10→1→11

✓ 用例3: nPosX 取1, nPosY取-1

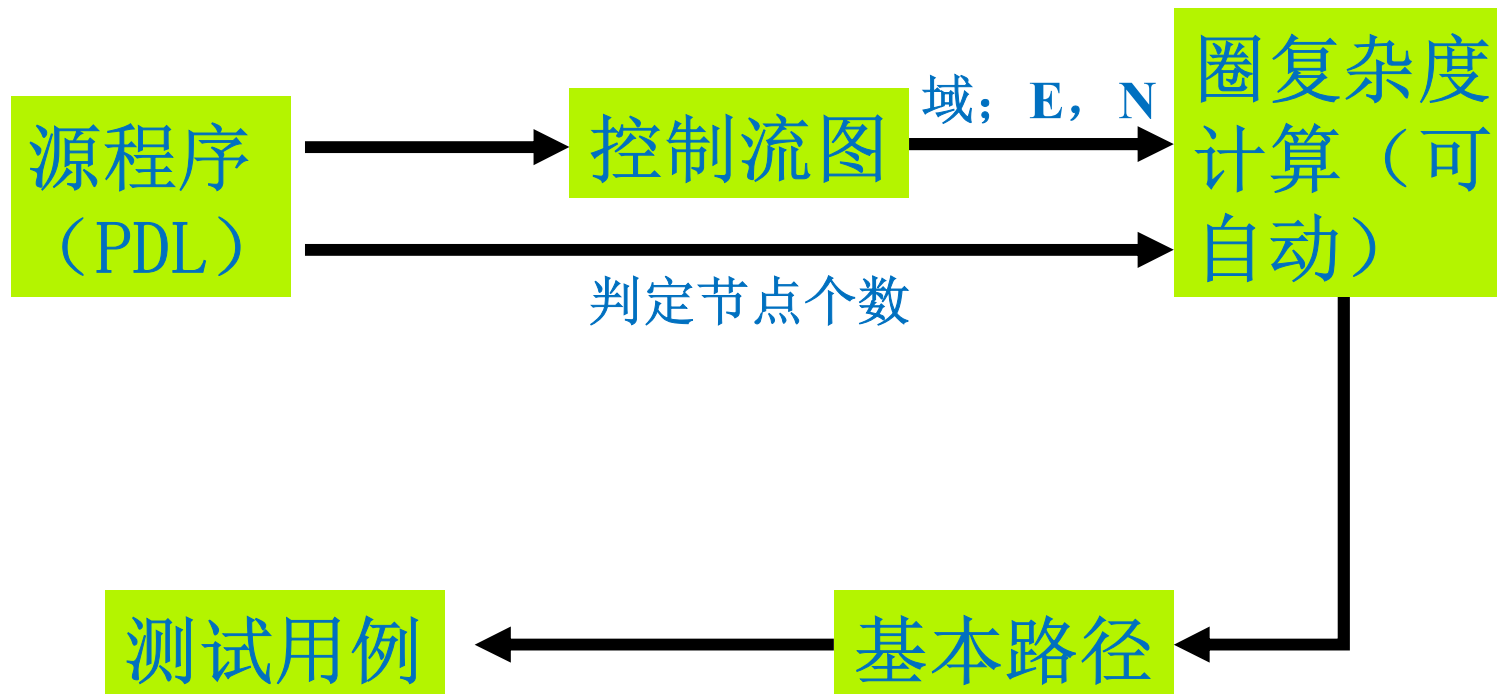
P₄: 1→2, 3→6→8→9→10→1→11

✓ 用例4: nPosX 取1, nPosY取-3



基本路径测试...

■ 测试用例设计过程一览

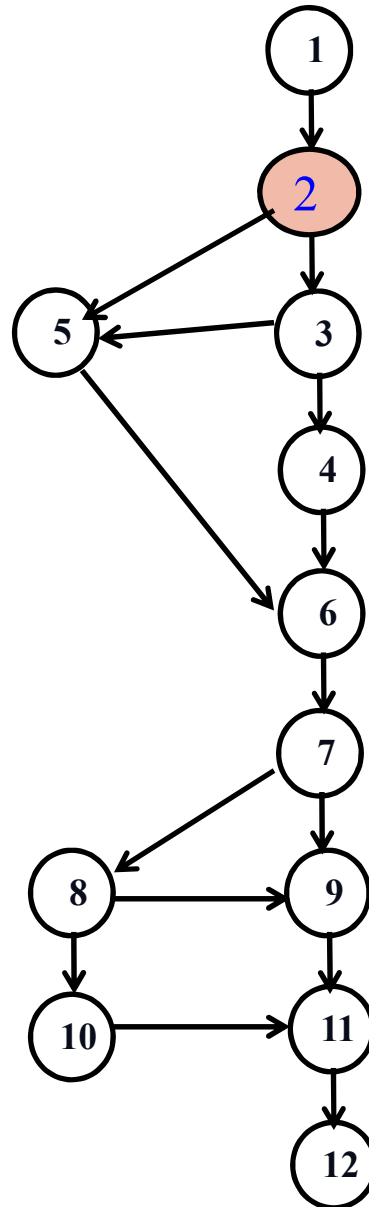


```

1:  start
    input (a,b,c,d)
2:  if (a>0)
3:      and (b>0)
4:  then x=a+b
5:  else x=a-b
6:  endif
7:  if (c>a)
8:      or (d<b)
9:  then y=c-d
10: else y=c+d
11: endif
12: print (x,y)
    stop

```

1. 流程图



2. 圈复杂度

$$V(G)=5$$

3. 基本路径

p1: 1,2,5,6,7,9,11,12

p2: 1,2,5,6,7,8,9,11,12

p3: 1,2,5,6,7,8,10,11,12

p4: 1,2,3,5,6,7,9,11,12

p5: 1,2,3,4,6,7,9,11,12

4. 测试用例

p1: 1,2,5,6,7,9,11,12

a=-1, b=1, c=1, d=1

p2: 1,2,5,6,7,8,9,11,12

a=-1, b=1, c=-1, d=0

p3: 1,2,5,6,7,8,10,11,12

a=-1, b=1, c=-1, d=1

p4: 1,2,3,5,6,7,9,11,12

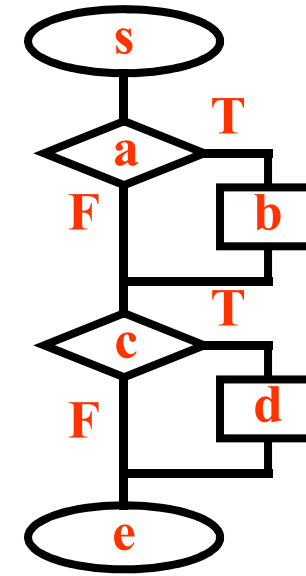
a=1, b=-1, c=2, d=1

p5: 1,2,3,4,6,7,9,11,12

a=1, b=1, c=2, d=1

练习：对下列子程序进行基本路径测试

```
procedure example(y,z:real;var x:real);  
begin  
  if (y>1) and (z=0) then x:=x/y;  
  if (y=2) or (x>1) then x:=x+1;  
end;
```



正确：sabcde、sabce、sace
或者sabcde、sacde、sace
或者sabcde、sabce、sacde

错误：sabcde、sabce、sacde、sace

基本路径测试...

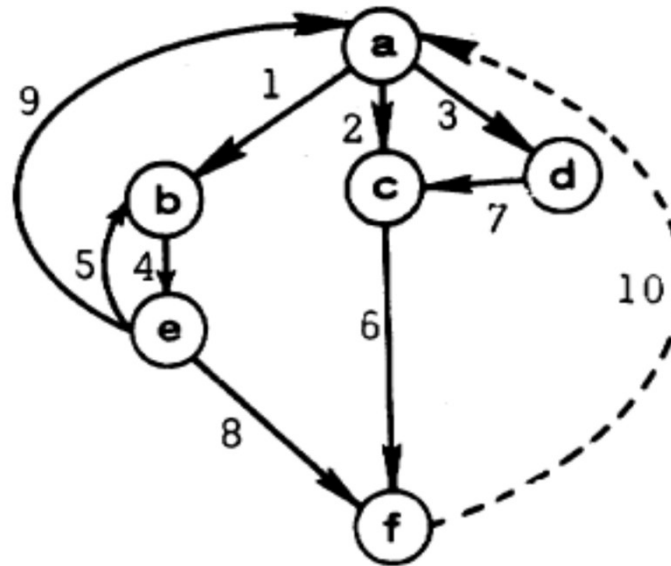
■基本路径的有什么特性？

每一条路径都可以表示为边的向量，即边作为向量的维；

$$P_x = (e_1, e_2, \dots, e_n), \quad n = \text{EdgeSize}$$

如果路径经过边1次，向量值对应的维就是1，如果经过2次，向量值就是2，以此类推。

■基本路径有什么特性？



任意给定一条路径: $P_0 = a \rightarrow b \rightarrow e \rightarrow b \rightarrow e \rightarrow b \rightarrow e \rightarrow f$

则它对应的向量为:

Edge:	1	2	3	4	5	6	7	8	9
P_0 :	1	0	0	3	2	0	0	1	0

基本路径测试...

■基本路径有什么特性？

基本路径是这样一组路径的集合，其它任何路径对应的向量，都可以用这些基本路径对应的向量通过线性运算组合出来。

基本路径测试...

■基本路径有什么特性？

我们就有如下一个基本路径集合：

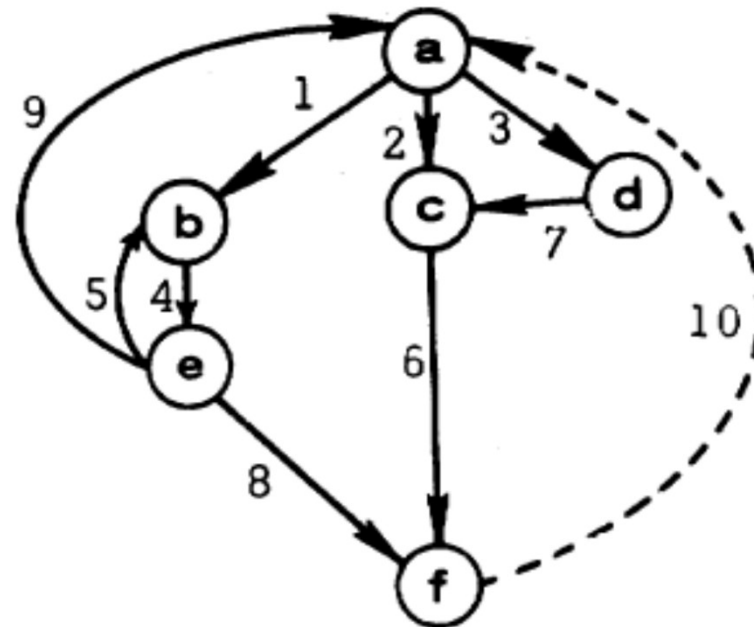
P_1 : a c f

P_2 : a d c f

P_3 : a b e f

P_4 : a b e b e f

P_5 : a b e a c f



基本路径测试...

■基本路径有什么特性？

对应的向量分别是：

Edge:	1	2	3	4	5	6	7	8	9
P_1 :	0	1	0	0	0	1	0	0	0
P_2 :	0	0	1	0	0	1	1	0	0
P_3 :	1	0	0	1	0	0	0	1	0
P_4 :	1	0	0	2	1	0	0	1	0
P_5 :	1	1	0	1	0	1	0	0	1

基本路径测试...

■基本路径有什么特性？

任何其它路径都可以通过基本路径向量组合运算出来：

例如路径

$$\mathbf{P}_0: \quad 1 \quad 0 \quad 0 \quad 3 \quad 2 \quad 0 \quad 0 \quad 1 \quad 0$$

$$\mathbf{P}_0 = x_1 \mathbf{P}_1 + x_2 \mathbf{P}_2 + x_3 \mathbf{P}_3 + x_4 \mathbf{P}_4 + x_5 \mathbf{P}_5$$

经过求解，我们可以得到： $\mathbf{P}_0 = 2\mathbf{P}_4 - \mathbf{P}_3$

基本路径测试...

■基本路径集合不唯一

基本路径集合的大小是由圈复杂度确定的，但基本路径的组成并不确定。

P_1 : a c f

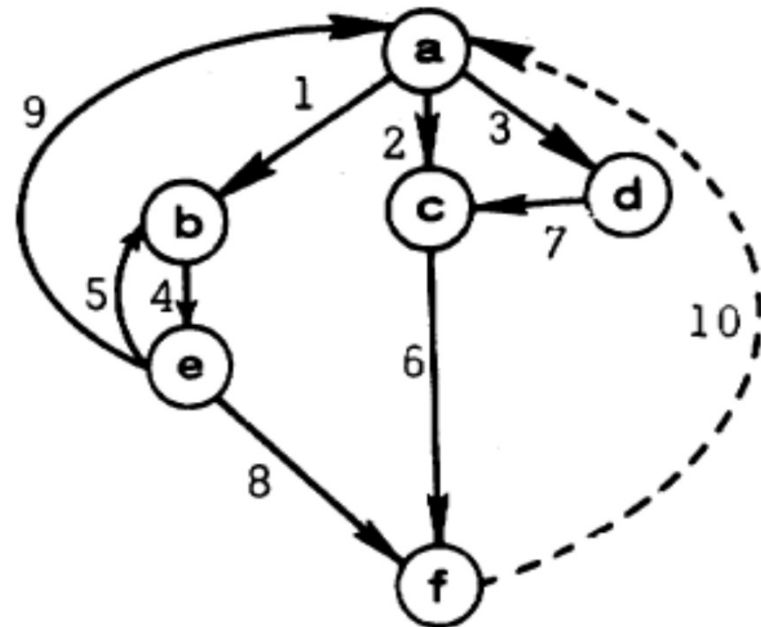
P_2 : a d c f

P_3 : a b e f

P_4 : a b e b e f

P_5 : a b e a c f

$P_{5_{\text{new}}}$: a b e a d c f



基本路径测试...

■基本路径集合不唯一

对应的向量是:

Edge:	1	2	3	4	5	6	7	8	9
P_1 :	0	1	0	0	0	1	0	0	0
P_2 :	0	0	1	0	0	1	1	0	0
P_3 :	1	0	0	1	0	0	0	1	0
P_4 :	1	0	0	2	1	0	0	1	0
P_5 :	1	1	0	1	0	1	0	0	1
P_{5new} :	1	0	1	1	0	1	1	0	1

基本路径测试...

■基本路径集合不唯一

P_0 : 1 0 0 3 2 0 0 1 0

$$\begin{aligned} P_0 &= x_1 P_1 + x_2 P_2 + x_3 P_3 + x_4 P_4 + x_5 P_{5\text{new}} \\ &= 2P_4 - P_3 \end{aligned}$$

P : abebeadcf

P : 1 0 1 2 1 1 1 0 1

$$P = P_{5\text{new}} + P_4 - P_3$$

$$P = P_5 + P_2 + P_4 - P_3 - P_1$$

基本路径测试...

■基本路径集合不唯一

新的基本路径集合和前面的集合相比，只是最后一个路径不同。但是这5条路径中都是线性无关的，而且，其它的路径都可以通过他们组合生成，因此也是基本路径集合。

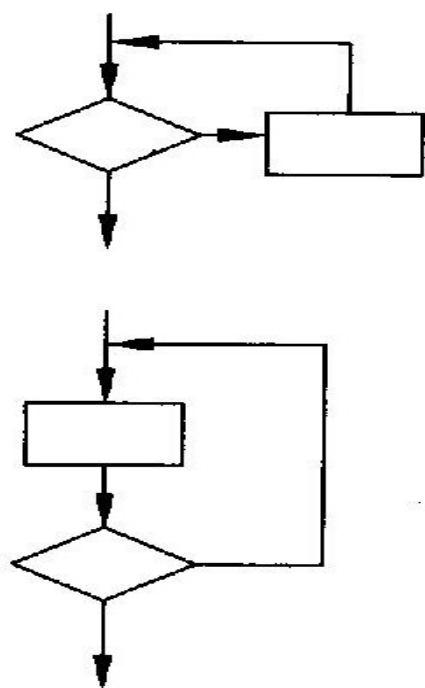
基于控制流的测试

—循环处理方法

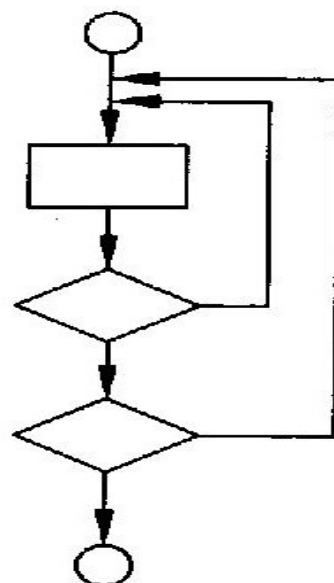
循环处理方法

循环分为4种不同类型：

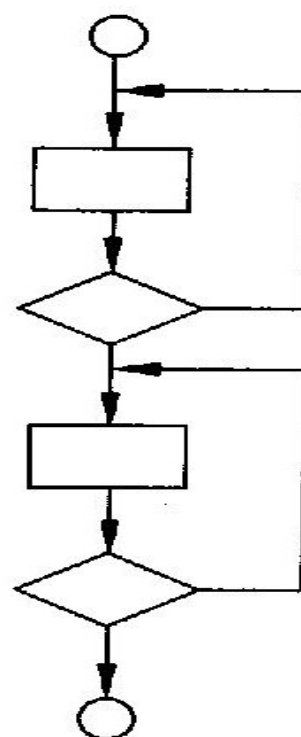
- 简单循环
- 嵌套循环
- 串接循环
- 非结构循环



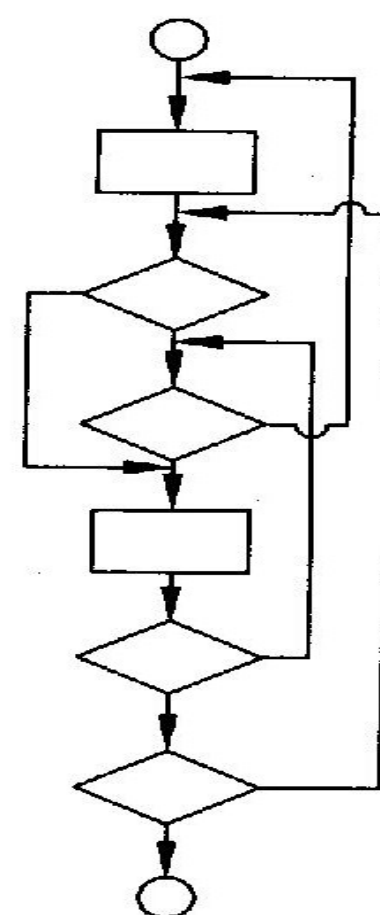
(a) 简单循环



(b) 嵌套循环



(c) 串接循环



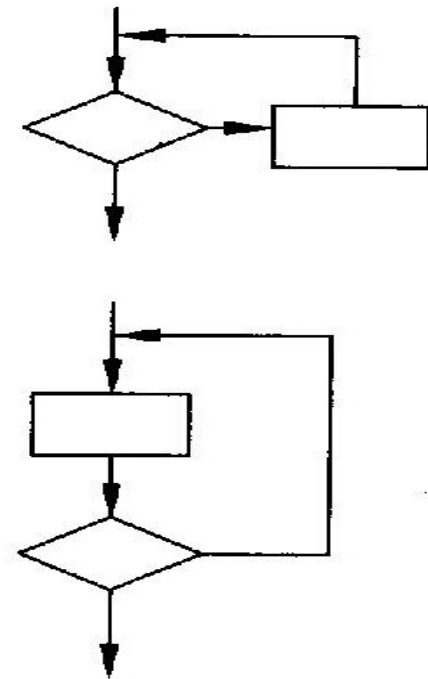
(d) 非结构循环

循环处理方法...

■ 简单循环的测试用例构造

- (1) 跳过整个循环 [执行0次循环]
- (2) 只执行1次循环
- (3) 执行2次循环
- (4) 执行 m ($m < n$) 次循环
- (5) 执行 $n-1$, n , $n+1$ 次循环

适用边界值测试的 $6n+1$ 公式



(a) 简单循环

循环处理方法...

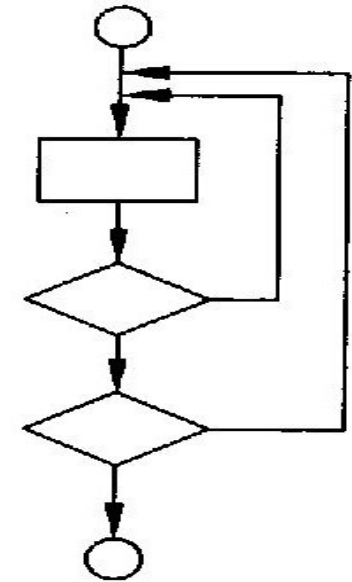
■ 嵌套循环的测试用例构造

① 先测试最内层循环：

- 所有外层的循环变量置为最小值
- 最内层按简单循环测试；

② 由里向外，测试上层循环：

- 此层以外的所有外层循环的循环变量取最小值
- 此层以内的所有嵌套内层循环的循环变量取“典型值”
- 该层按简单循环测试；

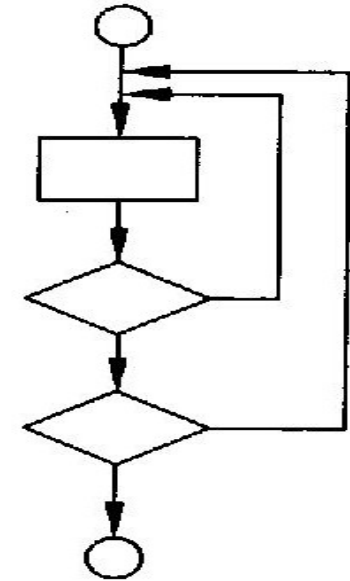


(b) 嵌套循环

循环处理方法...

■ 嵌套循环的测试用例构造...

- ③ 重复上一条规则，直到所有各层循环测试完毕；
- ④ 对全部各层循环同时取最小循环次数，或者同时取最大循环次数

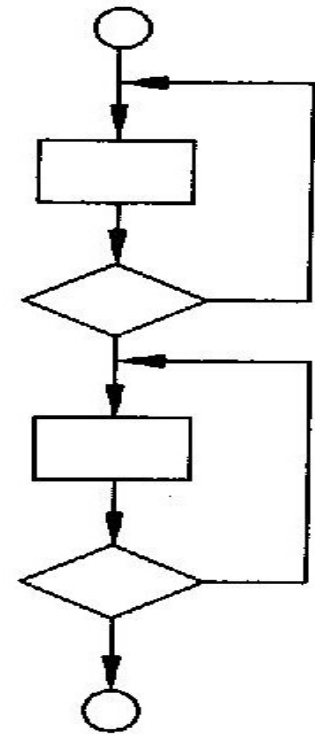


(b) 嵌套循环

循环处理方法...

■ 串接循环的测试用例构造

- ① 若串接的各个循环互相独立：
 - 分别用简单循环的方法进行测试；
- ② 若两个循环不独立（第一个循环的循环变量与第二个循环控制相关）
 - 把第一个循环看作外循环，第二个循环看作内循环，然后用测试嵌套循环的办法来处理。

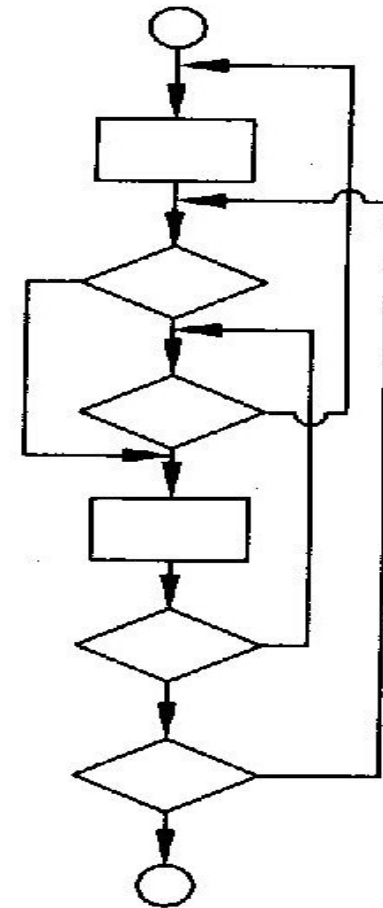


(c) 串接循环

循环处理方法...

■ 非结构循环

这一类循环应该先将其结构化，然后再测试。

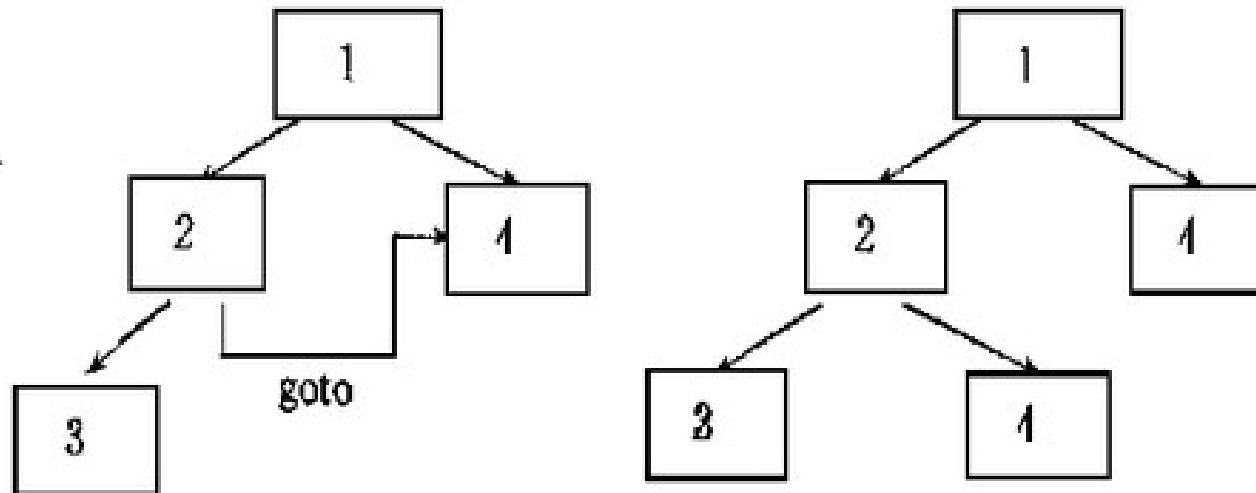


(d) 非结构循环

循环处理方法...

■非结构循环

例如：重复编码法



基于数据流的测试

主要内容

1. 什么是数据流测试？
2. 数据流测试和路径测试有什么区别？
3. 数据流测试基本思想
4. 数据流测试基本概念
5. 数据流测试覆盖标准

1 什么是数据流测试？

- **数据流**是变量的定义或使用顺序和变量的可能状态的一种抽象表示。
 - 变量的状态可以是：创建/定义(Creation/ Defined)、使用(Use)、清除/销毁(Killed/Destruction)。
 - 变量存在从创建、使用到销毁的一个完整状态变化过程。
- **数据流测试 (data-flow testing)** 是一种面向**变量定义位置-使用位置覆盖**的基于程序结构的测试方法。
 - 该测试方法重点关注**变量的定义与使用**。在选定的一组代码中搜索某个变量**所有的定义位置和使用位置**，并检查在程序运行时该变量的值将会如何变化，从而分析是否是 Bug 的产生原因。

2 数据流测试与路径测试有什么区别？

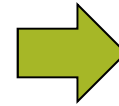
- 都是基于路径的测试，但“**路径**”的含义不完全相同
- **路径测试**是从控制流（逻辑覆盖）角度来分析的，面向**程序的潜在执行路径**设计一系列的覆盖标准用于衡量测试的覆盖率。
- **数据流测试**则是从数据流（变量的定义与使用覆盖）的角度来分析的。利用变量之间的关系，通过**变量的定义-使用路径**设计一系列的覆盖标准用于衡量测试的覆盖率。
- 实际上数据流测试可认为是路径测试的“**真实性**”检查，是对基于路径测试的一种改良。例如，通过数据流分析可以检查变量赋值错误。

3 数据流测试的基本思想

■ 数据流覆盖

- 程序是**对数据的加工处理过程**，对程序的测试可从**数据处理流程**的角度进行考虑。
- 数据处理流程**对应数据流图**
 - ◆ **数据流图**（DFG: data flow graph）类似于控制流图（CFG: control flow graph），描述了测试对象代码的处理过程，同时也详细描述了代码中**变量的创建、使用 and 销毁的状态**。通过检查数据流图来验证测试对象代码中**每个变量的状态组合是否正确**。
- 受**Debugging**过程的启发：寻找关于某**变量的定义位置和使用位置**，思考程序在运行时该变量的值会如何变化，从而分析Bug产生的原因。

control flow graph
CFG for control-flow testing



data flow graph
DFG for *data-flow testing*

注：DFD(data flow diagram)

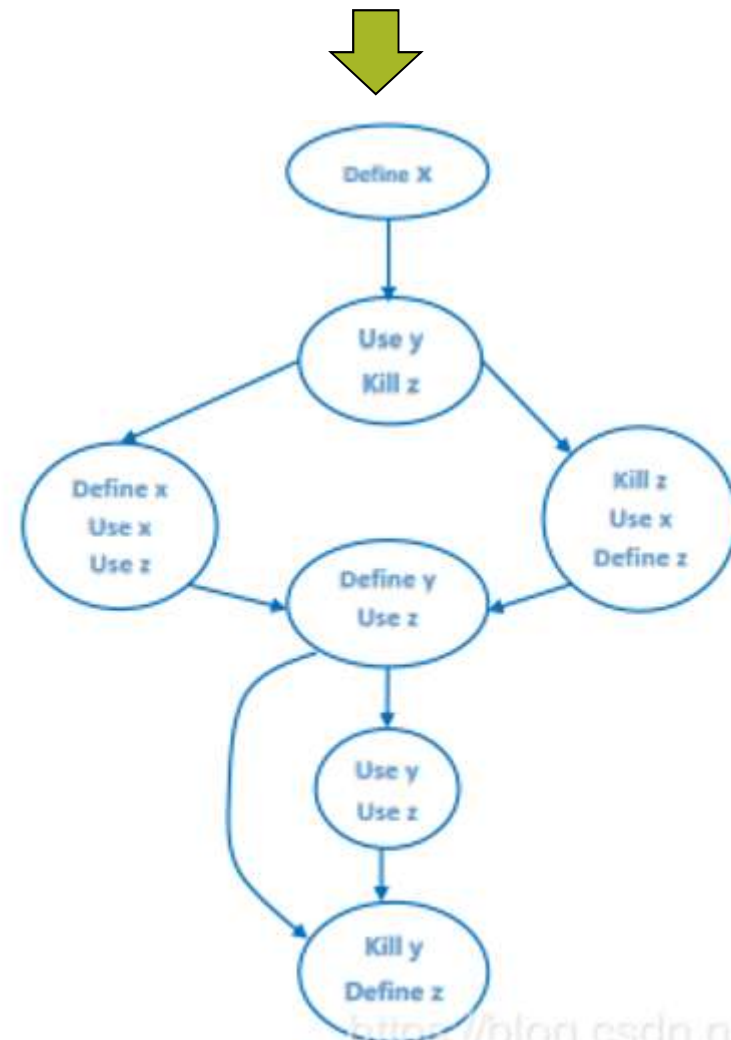
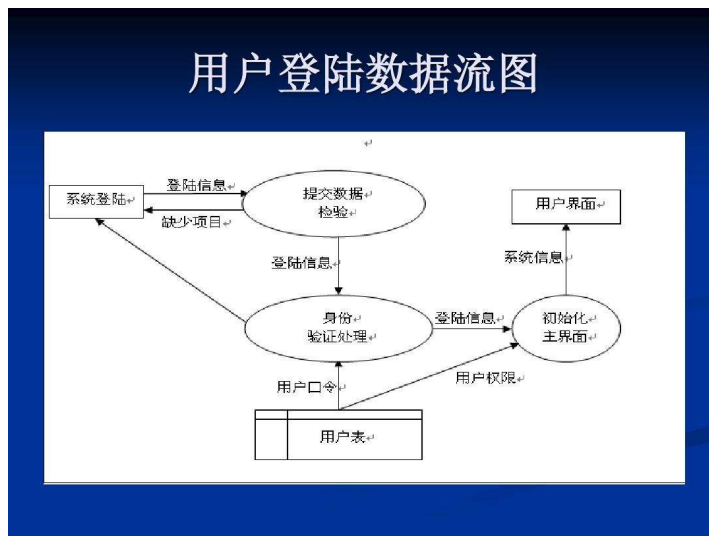
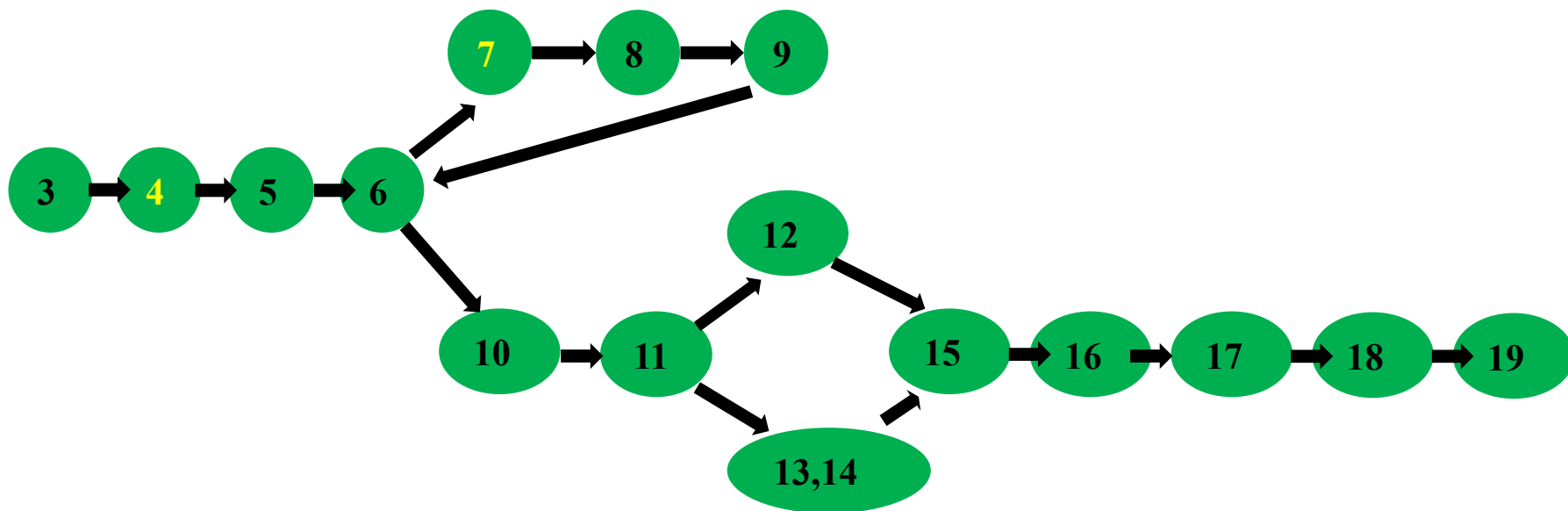


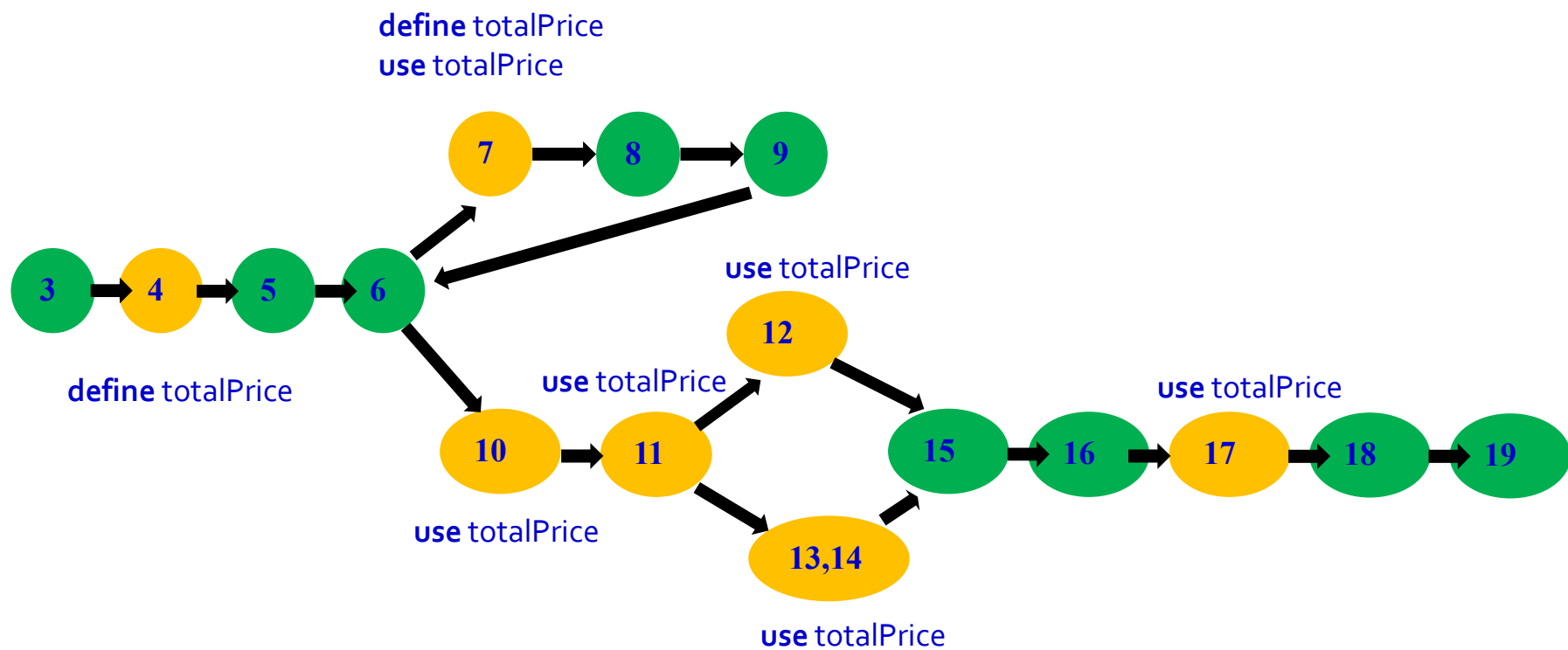
图 1 一个数据流图的例

示例

```
1  Program Example()
2  var staffDiscount, totalPrice, finalPrice, discount, price
3  staffDiscount = 0.1
4  totalPrice = 0
5  input(price)
6  while(price != -1) do
7    totalPrice = totalPrice + price
8    input(price)
9  od
10 print("Total price: " + totalPrice)
11 if(totalPrice>15.00) then
12   discount = (staffDiscount * totalPrice) + 0.50
13 else
14   discount = staffDiscount * totalPrice
15 fi
16 print ("Discount: " + discount)
17 finalPrice = totalPrice - discount
18 print ("Final price: " + finalPrice)
19 endProgram
```



程序控制流图CFG



程序数据流图DFG

- **数据流测试**的作用是用来测试变量定义位置和使用位置之间的路径。这些路径也称为“**定义-使用对**”（define-use pairs 或 du-pairs）或“**定义-使用路径**（define-use paths）”。通过数据流分析而生成的测试集（test suite）可用来获得**针对每个变量的“定义-使用路径”的 100%覆盖**。
- 要追踪整个程序代码中的**每个变量的定义和使用**时，并不需在测试时考虑被测对象的**控制流**。
- 根据程序中某个**变量的定义位置和使用位置**来选择程序的测试路径

4 数据流测试的基本概念

基本定义:

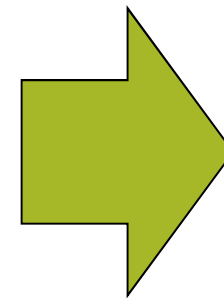
1.P——程序

2.G(P)——程序数据流图

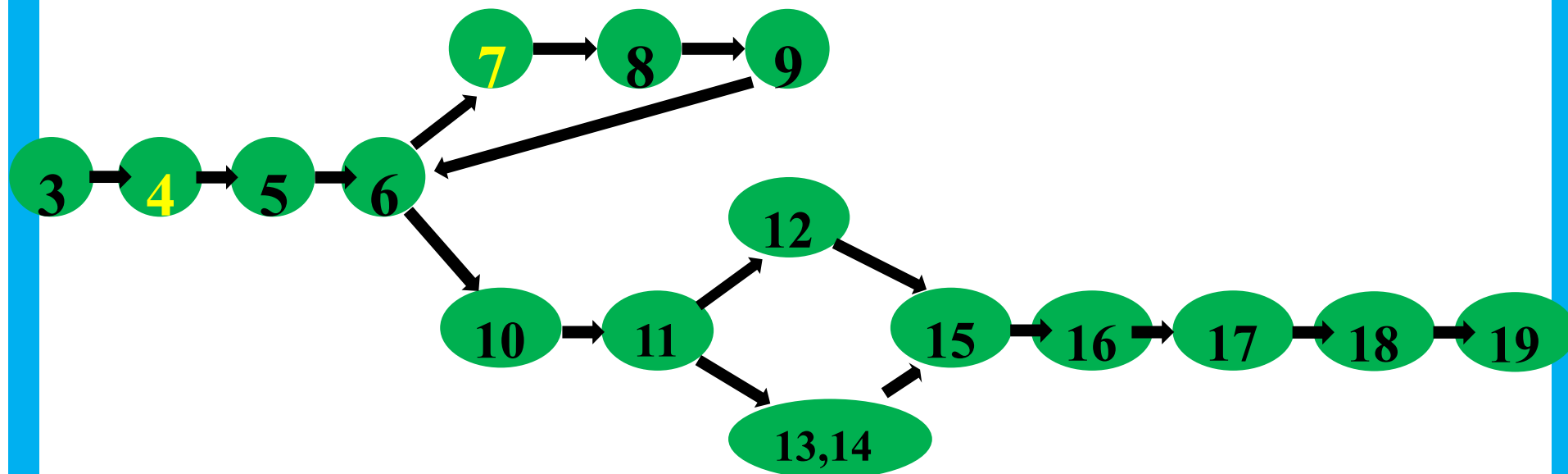
3.V——变量集合

4.PATH(P)——P的所有路径集合

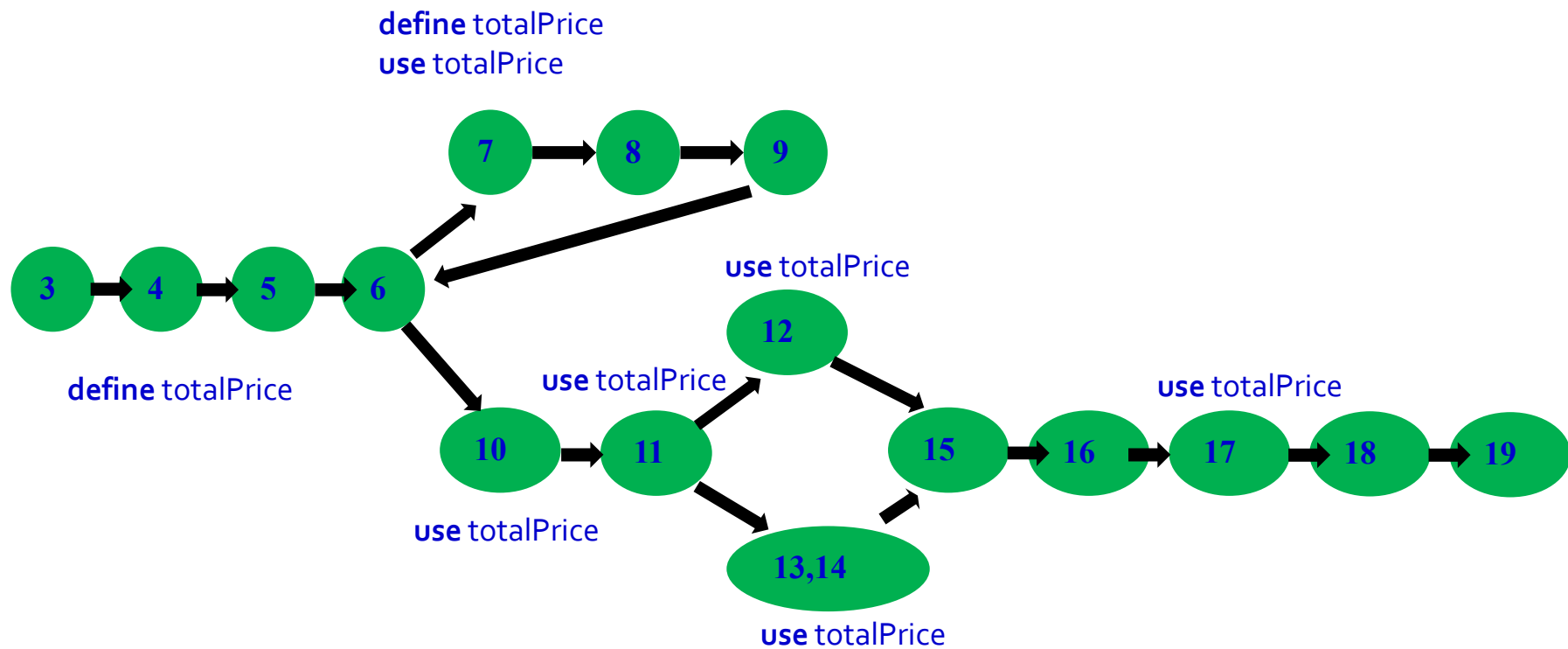
```
1  Program Example()
2  var staffDiscount, totalPrice, finalPrice, discount, price
3  staffDiscount = 0.1
4  totalPrice = 0
5  input(price)
6  while(price != -1) do
7    totalPrice = totalPrice + price
8    input(price)
9  od
10 print("Total price: " + totalPrice)
11 if(totalPrice > 15.00) then
12   discount = (staffDiscount * totalPrice) + 0.50
13 else
14   discount = staffDiscount * totalPrice
15 fi
16 print ("Discount: " + discount)
17 finalPrice = totalPrice - discount
18 print ("Final price: " + finalPrice)
19 endProgram
```



画出此程序的
CFG/DFG,
写出其变量集合



程序控制流图CFG



程序数据流图DFG

$V = \{\text{staffDiscount, totalPrice, finalPrice, discount, price}\}$

5.定义节点DEF(v,n)——在节点n定义了变量v,
即变量赋值语句

例如: `input x; x = 2;`

6.使用节点USE(v,n)——在节点n使用了变量v

例如: `print x; a=2+x;`

```
1  Program Example()
2  var staffDiscount, totalPrice, finalPrice, discount, price
3  staffDiscount = 0.1
4  totalPrice = 0
5  input(price)
6  while(price != -1) do
7    totalPrice = totalPrice + price
8    input(price)
9  od
10 print("Total price: " + totalPrice)
11 if(totalPrice>15.00) then
12   discount = (staffDiscount * totalPrice) + 0.50
13 else
14   discount = staffDiscount * totalPrice
15 fi
16 print ("Discount: " + discount)
17 finalPrice = totalPrice - discount
18 print ("Final price: " + finalPrice)
19 endProgram
```

写出此程序
中变量
totalPrice
的**定义位置**
(节点) 和
使用位置
(节点)

节点	类型	语句
4	D	totalPrice = 0
7	D	totalPrice = totalPrice + price
7	U	totalPrice = totalPrice + price
10	U	print("Total price: " + totalPrice)
11	U	if(totalPrice>15.00) then
12	U	discount = (staffDiscount * totalPrice) + 0.50
14	U	discount = staffDiscount * totalPrice
17	U	finalPrice = totalPrice - discount

7.谓词使用**P-use**——**USE(v,n)**位于一个谓词中,
即条件判断语句中

例如: if b>6

8.计算使用**C-use**——**USE(v,n)**位于一个计算中,
即计算表达式中

例如: x=3+b

C-use的进一步划分：

9. 输出使用O-use——变量值被输出到屏幕/打印机

10. 定位使用L-use——变量值用于定位数组位置（例如，a[b]中的b）

11. 迭代使用I-use——变量值用于控制循环次数

节点	类型	语句
7	C-Use	totalPrice = totalPrice + price
10	O-Use	print("Total price: " + totalPrice)
11	P-Use	if(totalPrice>15.00) then
12	C-Use	discount = (staffDiscount * totalPrice) + 0.50
14	C-Use	discount = staffDiscount * totalPrice
17	C-Use	finalPrice = totalPrice - discount

12.变量 v 的定义-使用路径(du-path):

给定 $PATH(P)$ 中的某条路径, 如果定义节点 $DEF(v,m)$ 为该路径的起始节点, 使用节点 $USE(v,n)$ 为该路径的终止节点, 则该路径是 v 的一条定义-使用路径(du-path)。

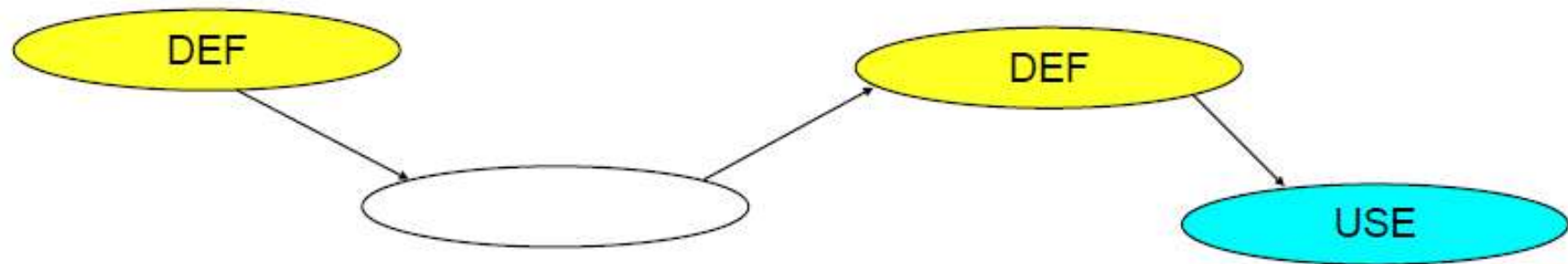
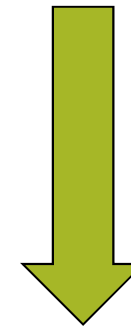


Figure 2: An example of a DU-path

```
1 Program Example()
2 var staffDiscount, totalPrice, finalPrice, discount, price
3 staffDiscount = 0.1
4 totalPrice = 0
5 input(price)
6 while(price != -1) do
7   totalPrice = totalPrice + price
8   input(price)
9 od
10 print("Total price: " + totalPrice)
11 if(totalPrice>15.00) then
12   discount = (staffDiscount * totalPrice) + 0.50
13 else
14   discount = staffDiscount * totalPrice
15 fi
16 print ("Discount: " + discount)
17 finalPrice = totalPrice - discount
18 print ("Final price: " + finalPrice)
19 endProgram
```

写出变量totalPrice的du路径



p1: 4, 5, 6, 7
p2: 4, 5, 6, 7, 8, 9, 10 或
4, 5, 6, 10
p3: 4,5,6,10,11
p4: 7,8,9,6,7
.....

13.变量v的定义-清洁路径(define-clear-path/dc-path):

如果变量v的某个定义-使用路径，除起始节点外没有其它定义节点，则该路径是变量v的**定义-清洁路径**（dc-path）。

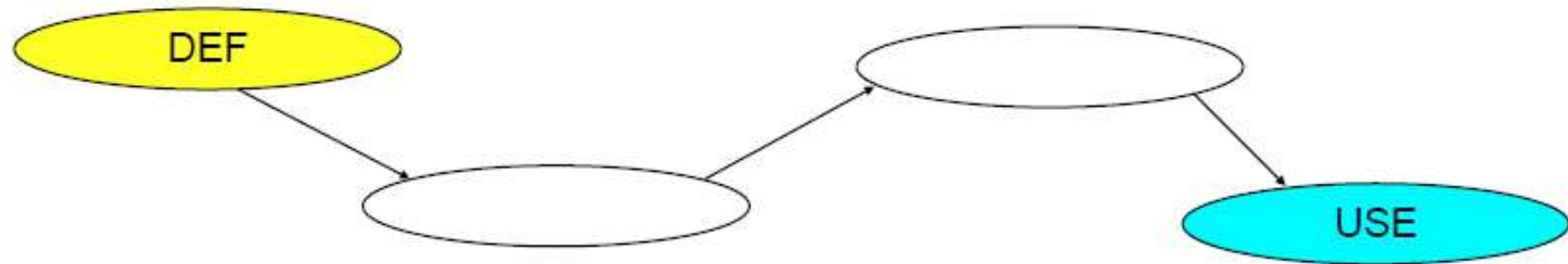


Figure 3: An example of a DU-path that is also definition-clear

```
1 Program Example()
2 var staffDiscount, totalPrice, finalPrice, discount, price
3 staffDiscount = 0.1
4 totalPrice = 0
5 input(price)
6 while(price != -1) do
7   totalPrice = totalPrice + price
8   input(price)
9 od
10 print("Total price: " + totalPrice)
11 if(totalPrice>15.00) then
12   discount = (staffDiscount * totalPrice) + 0.50
13 else
14   discount = staffDiscount * totalPrice
15 fi
16 print ("Discount: " + discount)
17 finalPrice = totalPrice - discount
18 print ("Final price: " + finalPrice)
19 endProgram
```

写出变量totalPrice
的dc路径？



p1: 4, 5, 6, 7
p2: 4, 5, 6, 10
.....

数据流覆盖测试步骤:

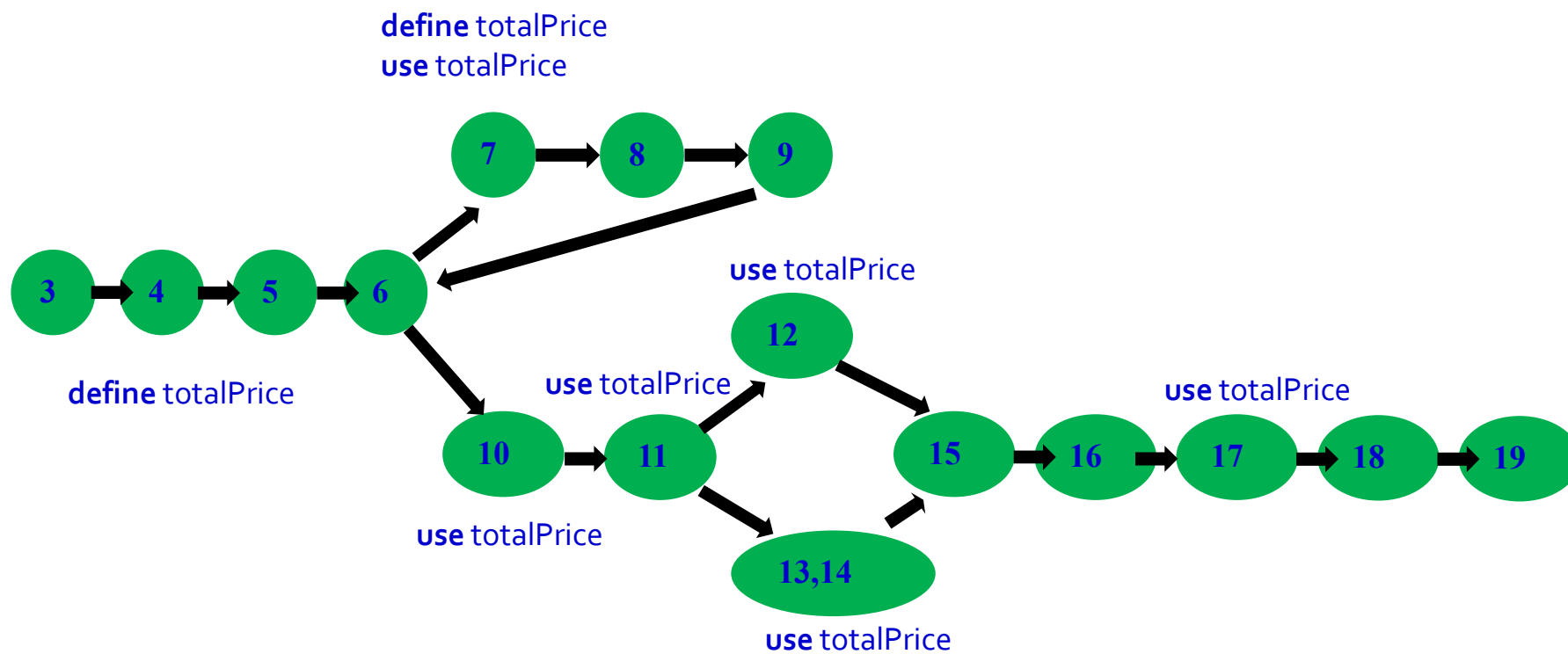
Step 1.对于给定的程序，构造相应的程序数据流图

Step 2.找出所有变量的定义-使用路径

Step 3.考察测试用例对这些路径的覆盖程度，即可作为衡量测试效果的度量值

```
1 Program Example()
2 var staffDiscount, totalPrice, finalPrice, discount, price
3 staffDiscount = 0.1
4 totalPrice = 0
5 input(price)
6 while(price != -1) do
7   totalPrice = totalPrice + price
8   input(price)
9 od
10 print("Total price: " + totalPrice)
11 if(totalPrice>15.00) then
12   discount = (staffDiscount * totalPrice) + 0.50
13 else
14   discount = staffDiscount * totalPrice
15 fi
16 print ("Discount: " + discount)
17 finalPrice = totalPrice - discount
18 print ("Final price: " + finalPrice)
19 endProgram
```

以变量totalPrice为例



程序数据流图DFG

数据流测试练习——以变量**totalPrice**为例

1. 分析定义节点和使用节点

Node	Type	Code
4	DEF	totalPrice = 0
7	DEF	totalPrice = totalPrice + price
7	USE	totalPrice = totalPrice + price
10	USE	print("Total price: " + totalPrice)
11	USE	if(totalPrice > 15.00) then
12	USE	discount = (staffDiscount * totalPrice) + 0.50
14	USE	discount = staffDiscount * totalPrice
17	USE	finalPrice = totalPrice - discount

1.1 分析并统计变量定义节点的出现次数： 2

1.2 分析并统计每次变量使用节点的类型和总次数： 6

数据流测试练习——以变量totalPrice为例

2. 列举可能的DU路径： $2 \times 6 = 12$

6 while(price != -1) do [条件成立]

p1: 4, 5, 6, 7

p2: 4, 5, 6, 7, 8, 9, 10

p3: 4, 5, 6, 7, 8, 9, 10, 11

p4: 4, 5, 6, 7, 8, 9, 10, 11, 12

p5: 4, 5, 6, 7, 8, 9, 10, 11, 13, 14

p6: 4, 5, 6, 7, 8, 9, 10, 11, 12, 15, 16, 17

p7: 7, 8, 9, 6, 7

p8: 7, 8, 9, 10

p9: 7, 8, 9, 10, 11

p10: 7, 8, 9, 10, 11, 12

p11: 7, 8, 9, 10, 11, 13, 14

p12: 7, 8, 9, 10, 11, 12, 15, 16, 17

6 while(price != -1) do [条件不成立]

p1: 4, 5, 6, 10

p2: 4, 5, 6, 10, 11

p3: 4, 5, 6, 10, 11, 12

p4: 4, 5, 6, 10, 11, 13, 14

p5: 4, 5, 6, 10, 11, 12, 15, 16, 17

数据流测试练习——以变量totalPrice为例

3 约简DU路径

p1: 4, 5, 6, 7

p2: 4, 5, 6, 7, 8, 9, 10

p3: 4, 5, 6, 7, 8, 9, 10, 11

p4: 4, 5, 6, 7, 8, 9, 10, 11, 12

p5: 4, 5, 6, 7, 8, 9, 10, 11, 13, 14

p6: 4, 5, 6, 7, 8, 9, 10, 11, 12, 15, 16, 17

p7: 7, 8, 9, 6, 7

p8: 7, 8, 9, 10

p9: 7, 8, 9, 10, 11

p10: 7, 8, 9, 10, 11, 12

p11: 7, 8, 9, 10, 11, 13, 14

p12: 7, 8, 9, 10, 11, 12, 15, 16, 17

经过约简得到**3条DU路径**

数据流测试练习——以变量totalPrice为例

4. 设计测试用例

p5: 4, 5, 6, 7, 8, 9, 10, 11, 13, 14

price = 14, -1

p6: 4, 5, 6, 7, 8, 9, 10, 11, 12, 15, 16, 17

price = 16, -1

p7: 7, 8, 9, 6, 7

price = 14, 16, -1

```
1 Program Example()
2 var staffDiscount, totalPrice, finalPrice, discount, price
3 staffDiscount = 0.1
4 totalPrice = 0
5 input(price)
6 while(price != -1) do
7     totalPrice = totalPrice + price
8     input(price)
9 od
10 print("Total price: " + totalPrice)
11 if(totalPrice>15.00) then
12     discount = (staffDiscount * totalPrice) + 0.50
13 else
14     discount = staffDiscount * totalPrice
15 fi
16 print ("Discount: " + discount)
17 finalPrice = totalPrice - discount
18 print ("Final price: " + finalPrice)
19 endProgram
```

以变量price为例

对于变量**price**,有两个定义节点和两个使用节点:

- Defining nodes:

- DEF(price, 5)
- DEF(price, 8)

- Usage nodes:

- USE(price, 6)
- USE(price, 7)

变量**price**对应4条**du-path**:

- $\langle 5, 6 \rangle$
- $\langle 5, 6, 7 \rangle$
- $\langle 8, 9, 6 \rangle$
- $\langle 8, 9, 6, 7 \rangle$

变量**price**对应的4条**du-path**同时也是**dc-path**

5 数据流测试常用的覆盖标准

□ 1. **Rapps**和**Weyuker**标准



Elaine Weyuker

Professor

Ph.D., Computer Science

Rutgers University, 1977

□ 2. Ntafos标准

□ 3. Ural标准

□ 4. Laski和Korel标准

目标:

- (1) 保证所选路径数目总是有限的/可被实际处理的;
- (2) 寻找一种系统化的测试路径选择方案, 以帮助**发现未知缺陷**;

Rapps和Weyuker标准

IEEE TRANSACTIONS ON SOFTWARE ENGINEERING, VOL. SE-11, NO. 4, APRIL 1985

367

Selecting Software Test Data Using Data Flow Information

SANDRA RAPPS AND ELAINE J. WEYUKER

Abstract—This paper defines a family of program test data selection criteria derived from data flow analysis techniques similar to those used in compiler optimization. It is argued that currently used path selection criteria, which examine only the control flow of a program, are inadequate. Our procedure associates with each point in a program at which a variable is defined, those points at which the value is used. Several test data selection criteria, differing in the type and number of these associations, are defined and compared.

Index Terms—Data flow, program testing, test data selection.

elements fulfill the chosen criterion, and then to find the input data which would cause each of the chosen paths to be selected.

Using such path selection criteria as the basis for test data selection criteria presents two distinct problems. Consider the strongest path selection criterion, *all-paths*, which requires that *all* program paths $p_1, p_2, \dots$ be selected. This effectively partitions the input domain D into a set of classes $D = \cup D[j]$ such that for every $x \in D$, $x \in D[j]$ if and only if executing the pro-

Rapps和Weyuker标准

1. **All-Paths:** 等价于路径覆盖
2. **All-Edges:** 等价于分支覆盖
3. **All-Nodes:** 等价于语句覆盖

Rapps和Weyuker标准

4. All-Defs: 选择的路径中每个变量的每个定义节点都有一条dc-path到达该定义的某个使用节点

检查每个变量的每个定义是否都有使用？

5. All-P-Uses: 选择的路径中每个变量的每个定义节点都有一条dc-path到达该定义的每个P-use节点

Rapps和Weyuker标准

6. All-P-Uses/Some C-Uses: 选择的路径中的每个变量的每个定义节点都有一条dc-path到达该定义的每个P-use节点;但如果没有可达P-use节点, dc-path应到达至少一个C-use节点

7. All-C-Uses/Some P-Uses: 选择的路径中的每个变量的每个定义节点都有一条dc-path到达该定义的每个C-use节点;但如果没有可达C-use节点, dc-path应到达至少一个P-use节点

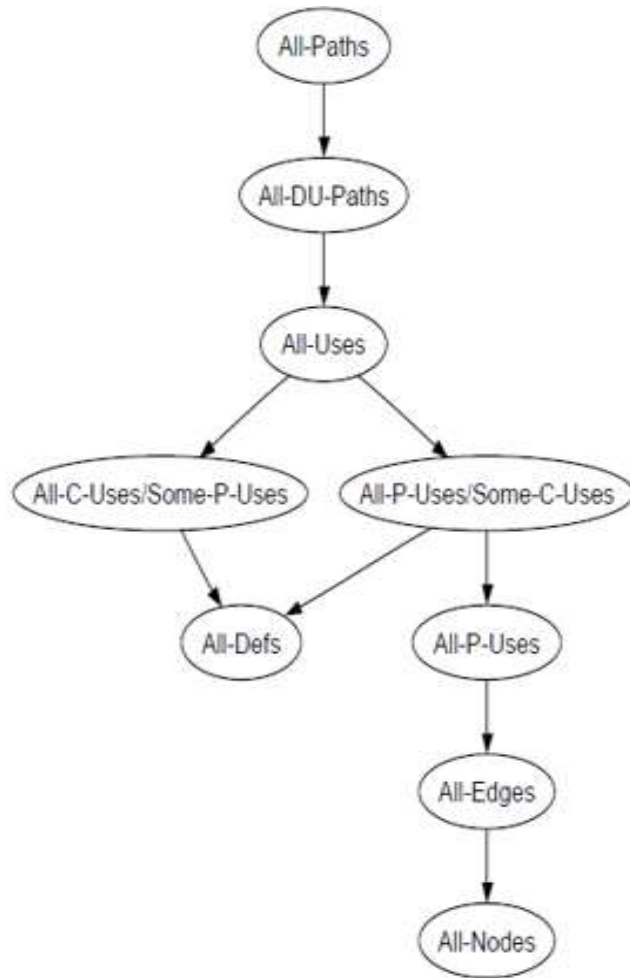
检查每个变量的每个定义是否都有使用? P使用或者C使用

Rapps和Weyuker标准

8. All-Uses: 选择的路径中的每个变量的每个定义节点都有一条dc-path到达该变量的每个使用节点, 若存在定义节点和使用节点间存在多条dc-path, 仅选取其中一条。

9. All-DU-Paths: 所有的DU-Path集合。路径中的每个变量的每个定义节点到达该变量的每个使用节点的所有dc-path。若存在循环, 仅取一个简单循环或者无循环。

Rapps-Weyuker覆盖9条标准的关系



拉普斯-韦约克（Rapps-Weyuker）数据流覆盖层次结构图如图所示。这个数据流覆盖标准层次结构图描述了所有的变量定义-使用路径情况。

实际上，这是变量的定义-使用路径情况的检查结构。当每个层次结构都检查完成，则表明针对变量的定义-使用路径达到100%的覆盖。

围绕定义-使用路径所得到的数据流测试覆盖标准包括：全路径、全定义-使用路径、全使用、全计算使用/部分谓词使用、全谓词使用/部分计算使用、全定义、全谓词使用、全边、全节点。

数据流测试...小结

- 数据流测试是在程序代码经过的路径上检查变量用法是否正确的一种方法。
- 可以发现“定义-使用”异常的缺陷。这里异常是指可能会导致程序失效的情形。例如，已经发现的数据流异常有：没有初始化就读取了变量的值，或根本没有使用变量的值。
- 异常可能会触发程序运行的风险，但并非数据流异常都会导致错误的程序行为发生(即失效)。因此对发现的数据流问题，需深入检查确定是否存在定义-使用问题。

数据流测试...

■ 优点

- (1) 揭示隐藏在代码变量定义和使用中的各种错误
- (2) 可以覆盖所有语句、所有分支、所有路径
- (3) 对代码的测试比较彻底

■ 缺点

- (1) 无法检测代码中不可达路径
- (2) 不验证需求规格

数据流测试...

■ 不可达路径 (infeasible path)

```
int f (int x, int y)
{
    return x+y;
    int z = x*y;
}
```

```
double x = sqrt(2);
if (x>5)
{
    xyz();
}
```

```
int n = 2+1;
if (4==n)
{
    xyz();
}
```